



Advanced prompt engineering

Systematic approaches to enhance LLM performance

Arttu Huttula

Master's thesis

June 2025

Master's Degree Programme in Artificial Intelligence and Data Analytics

Arttu, Huttula

Advanced prompt engineering. Systematic approaches to enhance LLM performance.

Jyväskylä: Jamk University of Applied Sciences, June 2025, 74 + 12 pages.

Degree Programme in Artificial Intelligent and Data Analytics. Master's thesis.

Permission for web publication: Yes

Language of publication: English

Abstract

A structured approach to prompt engineering was investigated to enhance the reliability, semantic quality, and consistency of outputs generated by large language models (LLMs). The study focused on designing and evaluating a systematic prompt optimization framework to address key challenges in current practices, such as inconsistent output behaviour, lack of repeatable prompting strategies and the absence of standardized evaluation metrics.

The research was grounded in the Design Science Research (DSR) methodology, enabling iterative development and empirical assessment of prompt strategies. Four prompting techniques – zero-shot, few-shot, chain-of-thought, and structured prompting – were applied across three natural language processing (NLP) tasks: summarization, sentiment classification, and paraphrasing. Controlled experiments were conducted using OpenAI's GPT-3.5-turbo and GPT-4o models. A total of 761 outputs were generated and evaluated using automated metrics, including BLEU, ROUGE-L and METEOR.

Findings revealed that structured prompts consistently outperformed alternative strategies in terms of semantic coherence and token efficiency. METEOR was identified as the most sensitive and discriminative metric across tasks. Beyond performance, the study explored the ethical implications of prompt design, including issues of bias, hallucinations and prompt injection risks. It was observed that prompt phrasing has a direct impact on fairness, interpretability and output safety.

The results support the conclusion that prompt engineering is transitioning from an experimental, ad hoc technique into a methodology-driven discipline. Systematic prompt construction and evaluation are essential for guiding LLM behaviour, improving reproducibility and reliable AI-generated content across diverse NLP applications.

Keywords/tags (subjects)

Prompt engineering, large language models, zero-shot, few-shot, chain-of-thought, structured prompt, evaluation metrics, natural language processing

Miscellaneous (Confidential information)

-

Arttu, Huttula

Advanced prompt engineering. Systematic approaches to enhance LLM performance

Jyväskylä: Jyväskylän ammattikorkeakoulu. Kesäkuu 2025, 74 + 12 sivua.

Tekoälyn ja data-analytiikan tutkinto-ohjelma. Opinnäytetyö YAMK.

Julkaisun kieli: englanti

Julkaisulupa avoimessa verkossa: kyllä

Tiivistelmä

Kehotteiden suunnittelun yhteydessä tutkittiin strukturoitua lähestymistapaa suurten kielimallien (LLM) tuottamien vastausten luotettavuuden, semanttisen laadun ja johdonmukaisuuden parantamiseksi. Tutkimuksessa kehitettiin ja arvioitiin systemaattinen kehotteiden optimointikehys, jonka tavoitteena oli vastata nykykäytäntöjen keskeisiin haasteisiin, kuten epäjohdonmukaisiin mallivastauksiin, toistettavien keho-testrategioiden puutteeseen ja arviointikriteerien standardoinnin puutteeseen.

Tutkimus toteutettiin Design Science Research (DSR) metodologian mukaisesti, joka mahdollisti kehotestrategioiden iteratiivisen kehittämisen ja empiirisen arvioinnin. Neljää kehotetyyppiä – nollanäyteoppiminen, vähänäyteoppiminen, ajatusketju ja rakenteinen kehote – sovellettiin kolmeen luonnollisen kielen käsittelyyn (NLP) tehtävään: tiivistäminen, mielipiteenluokittelu ja parafraasien tuottaminen. Kontrolloidut kokeet suoritettiin OpenAI:n GPT-3.5-turbo ja GPT-4o malleilla. Yhteensä tuotettiin 761 vastausta, jotka arvioitiin automatisoiduilla mittareilla (BLEU, ROUGE-L ja METEOR).

Tulokset osoittivat, että rakenteinen kehote tuotti johdonmukaisesti parempia tuloksia semanttisen laadun ja merkkitehokkuuden osalta verrattuna muihin strategioihin. METEOR-mittari osoittautui herkimmäksi ja erotteluvimmaksi arviointityökaluksi. Lisäksi tutkimuksessa tarkasteltiin kehotteiden suunnittelun eettisiä ulottuvuuksia, kuten vinoumia, hallusinaatioita ja kehoteinjektion riskejä. Havaittiin, että kehotteen muotoilu vaikuttaa suoraan oikeudenmukaisuuteen, tulkittavuuteen ja vastausten turvallisuuteen.

Tutkimuksen tulokset tukevat johtopäätöstä, että kehotteiden suunnittelu on siirtymässä kokeellisesta, ad hoc -lähestymistavasta kohti metodologialähtöistä tieteenalaa, jossa systemaattinen kehotteiden rakentaminen ja arviointi ovat keskeisiä keinoja ohjata kielimallien toimintaa ja varmistaa laadukkaat tekoäly vastaukset eri NLP-tehtävissä.

Avainsanat (asiasanat)

Kehotteiden suunnittelu, suuret kielimallit, nollanäyteoppiminen, vähänäyteoppiminen, ajatusketju, rakenteinen kehote, arviointimittarit, luonnollisen kielen käsittely

Muut tiedot (salassa pidettävät liitteet)

-

Contents

1	Introduction.....	5
2	Research ideology.....	6
2.1	Research questions and objectives.....	6
2.2	Problem identification and research hypothesis	6
2.3	Scope and limitations	7
2.4	Structure of the thesis	8
2.5	Use of AI in the thesis process.....	8
3	Theoretical background	9
3.1	Prompt engineering and its role in natural language processing.....	9
3.2	Large language models and their mechanisms.....	11
3.2.1	Autoregressive modelling and fine-tuning	12
3.2.2	GPT-4 specifics	13
3.3	Transformer networks and attention mechanisms	14
3.4	Evaluation metrics for prompt performance	15
3.4.1	Task success and accuracy.....	16
3.4.2	Quality metrics of natural language processing	16
3.4.3	Emerging automated metrics.....	17
3.4.4	Human evaluation.....	18
3.5	Overview of learning paradigms in prompt engineering	19
3.5.1	From Zero-shot to Chain-of-Thought	20
3.5.2	Prompt tuning and prefix tuning	21
3.5.3	In-context learning vs. fine-tuning	22
3.5.4	Safety and adversarial prompting	23
4	Structured design principles in prompt engineering.....	24
4.1	Semantic processing and prompt construction	24
4.2	Context sensitivity in AI responses	26
4.3	Systematic approaches to prompt optimization.....	27
4.3.1	Enumeration and grid search	28
4.3.2	Automated prompt generation	28
4.3.3	Gradient-based optimization and soft prompts	29
4.3.4	Reinforcement learning (RL) for prompts.....	29
4.3.5	LLM-based optimization.....	29
4.3.6	Human-in-the-loop and prompt engineering	30

4.4	Ethical considerations in prompt engineering	31
4.4.1	Bias and fairness	31
4.4.2	Hallucinations and misinformation	33
4.4.3	Security and manipulation risks	35
5	Research methodology	36
5.1	Design Science Research as a methodology	36
5.2	Prompt evaluation setup and environment.....	38
5.3	Evaluation strategy and workflow using BLEU, ROUGE and METEOR.....	38
6	Implementation and artifact development	39
6.1	Developing a systematic prompt optimization framework	39
6.2	Testing structured prompt variants.....	41
6.2.1	Structured prompt construction	42
6.2.2	Prompt comparison.....	43
6.3	Design and evaluation of in-context learning strategies.....	44
6.3.1	Few-shot prompting.....	44
6.3.2	Chain-of-Thought prompting	45
6.3.3	Evaluation and comparison	46
6.4	Environment setup and authentication.....	47
6.4.1	Experiment control and model switching.....	47
6.4.2	Execution workflow.....	49
6.4.3	Implementation of automated metric-based evaluation	50
6.4.4	Logging and score tracking	50
6.5	Evaluating the framework on real-world natural language processing tasks	51
6.5.1	Use case 1: Summarization	52
6.5.2	Use case 2: Sentiment classification.....	52
6.5.3	Use case 3: Paraphrasing.....	53
6.5.4	Dataset format and test distribution.....	53
7	Results and analysis.....	54
7.1	Task design and example use cases.....	54
7.2	Metric-based evaluation of prompt strategies	56
7.2.1	Use case 1: Text summarization.....	57
7.2.2	Use case 2: Sentiment classification.....	59
7.2.3	Use case 3: Text paraphrasing.....	61
7.3	Statistical evaluation and key findings.....	64

8	Conclusions.....	68
8.1	Summary and design science contributions	68
8.2	Research limitations and areas for development	69
8.3	Future research directions.....	70
	References	71
	Appendices	75
	Appendix 1. Dataset 1	75
	Appendix 2. Dataset 2	77
	Appendix 3. Dataset 3	82

Figures

Figure 1.	The relative strengths of fine-tuning and prompt engineering in five dimensions.....	11
Figure 2.	Causal attention in GPT-style transformer	15
Figure 3.	Prompt sensitivity example showing how different prompt phrasing impact model accuracy on reasoning tasks.....	25
Figure 4.	Demonstration of the “lost in the middle” effect for a GPT-3.5-turbo model on a question-answering task with 20 documents of context	26
Figure 5.	Illustration of a prompt optimization pipeline.....	30
Figure 6.	Hallucinated Chain of Thought - how a plausible but factually incorrect reasoning process can occur due to a misleading prompt.....	34
Figure 7.	A visual example of prompt injection	36
Figure 8.	Architecture of the prompt optimization framework.....	40
Figure 9.	Structured prompt	42
Figure 10.	Few-shot prompting	45
Figure 11.	Chain-of-Thought prompting.....	46
Figure 12.	Client object authentication	47
Figure 13.	Model switching	47
Figure 14.	API call	48
Figure 15.	Experiment loop	49
Figure 16.	Hugging Face evaluate.....	50
Figure 17.	Example jsonl structure	51
Figure 18.	Test input	53
Figure 19.	Evaluation metric average scores per task	56

Figure 20. Token efficiency vs. summarization quality	58
Figure 21. METEOR by prompt type (classification)	60
Figure 22. Token usage against METEOR score (classification)	61
Figure 23. METEOR by prompt type (paraphrasing)	62
Figure 24. Token usage vs. METEOR score (paraphrasing)	63
Figure 25. Average BLEU, ROUGE, METEOR per prompt type (paraphrasing)	64
Figure 26. Cross-task comparison of strategies	65
Figure 27. METEOR per token	66
Figure 28. Correlation between BLEU, ROUGE and METEOR	67

Tables

Table 1. Research questions of the thesis	6
Table 2. Prompting strategies and their effects on bias in LLM responses	32
Table 3. Application of Hevner's Design Science Research guideline	37
Table 4. Prompt strategy comparison in classification task	43
Table 5. Summary use case example	52
Table 6. Classification use case example	52
Table 7. Paraphrasing use case example	53
Table 8. Summarization prompting techniques and examples	55
Table 9. Average evaluation metrics for summarization by prompt type	57

1 Introduction

Large language models (LLMs) such as OpenAI's GPT-3.5-turbo and GPT-4 omni or 4o, have transformed the way organizations approach natural language processing (NLP) tasks, from text summarization and classification to conversational instances and decision support tools. While these models have significant creative capabilities, the quality, relevance and consistency of their results depend heavily on how users ask them questions. Prompt engineering – the practice of crafting instructions or queries to produce desired responses – has emerged as a key technique for guiding and optimizing LLM behaviour without requiring access to the model's internal or fine-tuning procedures.

Despite the growing popularity of prompt engineering, its application often remains ad hoc, experimental and difficult to replicate. Many users rely on a trial-and-error approach, with little systematic guidance on how to design effective prompts for different tasks or how to assess their performance in a controlled and measurable way. This lack of structure poses a barrier to both research and practical implementation, where repeatability, performance benchmarks and prompt generalization across tasks are increasingly important.

At the same time, new prompt strategies, such as zero-shot, few-shot and chain-of-thought prompting have been proposed and studied, showing promise in enhancing model performance for reasoning, classification, and multi-step tasks. However, their comparative productivity remains task-specific and context-sensitive, with few studies offering a unified framework for designing, testing and evaluating such prompt strategies systematically.

This thesis addresses this gap by proposing and implementing a systematic prompt optimization framework. The framework is designed to support structured experimentation with prompt strategies across various NLP tasks, using reproducible metrics such as BLEU, ROUGE and METEOR. The study is conducted following the Design Science Research (DSR) methodology, which emphasizes the development and evaluation of purposeful artifacts, in this case, prompt templates and an experimental testing pipeline. Using this approach, the thesis aims to bring both practical tools and methodological insights to the evolving prompt engineering.

2 Research ideology

2.1 Research questions and objectives

The primary objective of this thesis is to investigate how prompt engineering strategies influence the performance of LLMs across a variety of tasks. To achieve this goal, the study formulates one main research question and two supplementary questions that guide theoretical analysis, framework design and empirical evaluation. These questions address the utility of specific prompting strategies, the application of in-context learning methods and the use of automated evaluation metrics. Together, these research questions shape the direction of the methodology and determine the scope of experiment conducted through the prompt optimization framework. The questions are presented in Table 1.

Table 1. Research questions of the thesis

Main question	What is the impact of different prompt engineering strategies (structured, few-shot, chain-of-thought and zero-shot) on the quality and consistency of LLM-generated outputs?
Sub-question 1	How do different in-context learning strategies influence prompt optimization outcomes in large language models?
Sub-question 2	Which quantitative evaluation metrics best capture the effectiveness of different prompt strategies in LLMs?

2.2 Problem identification and research hypothesis

This section expands on the research questions and objectives by identifying the key challenges that limit the reliable and consistent use of prompt engineering strategies in LLMs:

- The absence of a systematic framework for designing effective prompts.
- High variability in model outputs in response to minor changes in prompt phrasing
- A lack of standardized evaluation criteria to compare the quality of different prompt strategies.

These issues form the foundation for the empirical work conducted in the latter part of this thesis. Based on problem framing, the following hypothesis is proposed:

The application of a structured prompt engineering framework, grounded in Design Science Research principles, will lead to more consistent and effective outputs from large language models, as measured by BLEU, ROUGE-L and METEOR scores.

The hypothesis is evaluated through multiple controlled evaluations involving varied prompt types sent to both GPT-3.5-turbo and GPT-4o using OpenAI Application Programming Interface (API).

2.3 Scope and limitations

This study focuses on the design, implementation and evaluation of prompt engineering strategies for LLMs, with a particular emphasis on zero-shot, few-shot, chain-of-thought and template-based also called structured prompting. The research is conducted using OpenAI’s GPT-3.5-turbo and GPT-4o models through API-based access, which enables controlled prompt testing and output retrieval for experimentation. The scope includes a series of natural language processing tasks – such as summarization, sentiment classification and paraphrasing – selected for their relevance and variety in prompt structure requirements. Evaluation is performed using widely known automated metrics (BLEU, ROUGE, METEOR) with the aim of measuring output quality, consistency and performance differences across prompt types.

However, the study has several limitations. First, it is limited to use of proprietary LLMs, which means that the internal mechanisms and training data of GPT models are not accessible or modifiable. As a result, prompt performance can only be assessed based on model outputs, not internal reasoning or token-level attention behaviour. Second, the assessment focuses on quantitative metrics and does not include human-in-the-loop assessment, which can be important for assessing coherence, creativity or nuanced language understanding. Third, the experiments are limited to English language tasks, and multilingual performance is not examined. Furthermore, due to resource and scope constraints, advanced optimization techniques such as reinforcement learning or dynamic prompt tuning are excluded from the implementation and are discussed only from a theoretical perspective.

Despite these limitations, the scope is still sufficient to support meaningful insight of the design and testing of prompt engineering strategies. The results are expected to inform future applications of prompt design in real-world use cases and guide further research on prompt optimization methods.

2.4 Structure of the thesis

This thesis follows a structured approach in which theoretical insights into prompt engineering guide the development of practical implementation. The work begins with an explanation of the motivation, research questions and methodological choices guiding the research. From there, a theoretical framework is developed to explore core concepts of prompt engineering, including the development of prompting strategies, semantic design models and the role of context in large language models. Building on this, the thesis adopts a Design Science Research approach to systematically guide the prompt optimization framework. This framework is implemented and applied in a controlled experimental setup that evaluate prompt variations on selected NLP tasks. The results are analysed using automated evaluation metrics to assess the effectiveness of different strategies. Finally, the thesis concludes with a discussion of key findings, limitations and future directions in prompt engineering research and practice.

Chapters 3 and 4 introduce core concepts such as zero-shot, few-shot and chain-of-thought prompting, as well as semantic prompt structuring strategies. These form the basis for design of a systematic prompt optimization framework, which is presented in Chapter 5. While advanced strategies such as reinforcement learning or automated prompt tuning are acknowledged, they are discussed as potential extensions rather than part of the implementation. This design is in line with the Design Science Research methodology used throughout the thesis, where both academic literature and practical considerations inform the creation and evaluation of a purposeful artifact.

2.5 Use of AI in the thesis process

During the writing process, generative AI tools were used to support clarity of language, sentence structure, and consistency of academic expression. These tools were applied as part of the revision process to refine grammar and improve fluency without changing the original content or argumentation. In the early stages of the work, AI was also utilized in planning the structure of the thesis

and to support the mapping of sources. In addition, AI was used to compare the tasks assigned in accordance with Jamk University of Applied Sciences 2024 assessment criteria to support the assessment of one's own expertise. The use of AI was limited to editorial and technical assistance, and all intellectual contributions, analyses, and conclusions presented in this thesis are the author's own.

3 Theoretical background

3.1 Prompt engineering and its role in natural language processing

Prompt engineering refers to the process of designing and phrasing natural language inputs that guide large language models (LLMs) to produce desired results. Unlike traditional supervised learning, which trains a model to map inputs x to outputs y by updating its parameters, prompt-based learning reframes x into a prompt x' , which the model executes using its pretrained internal knowledge (Liu et al., 2021). The result is then interpreted as the output y . This strategy eliminates the need for task-specific fine-tuning and has emerged as a transformative paradigm in natural language processing (NLP). Its popularity has exploded since the introduction of GPT-3 (Brown et al., 2020), which demonstrated that large-scale models can be generalized to new tasks using only well-crafted prompts. Since then, prompt engineering has developed into its own research area and there has been active research into how to construct prompts that produce accurate, safe and interpretable responses (Liu et al., 2021; Brown et al., 2020).

Prompt engineering plays a fundamental role in zero-shot and few-shot learning paradigms. In zero-shot prompting, a language model is presented with only a command—no examples—requiring it to rely entirely on its pre-trained knowledge to perform the task (Liu et al., 2021). Few-shot prompting improves on this setup by including a small number of examples (e.g., input–output pairs), that serve as demonstrations from which the model infers the structure of the task. Brown et al.'s (2020) GPT-3 study showed that with even 10 – 100 carefully designed examples, a 175 billion-parameter model could provide competitive performance on translation, question answering and arithmetic tasks without parameter updates.

This behaviour, called in-context learning, emphasizes the model's ability to generalize from textual context alone. Later studies have extended this to instruction-based prompting, where models are guided by natural language commands or goals (Ouyang et al., 2022). Fine-tuning the prompts, fine-tuning a large set of prompt response pairs further improved the ability to follow instructions and generate the desired outputs. For GPT-3.5-turbo and GPT-4o, prompt engineering is particularly impressive: even small changes in wording or order can lead to significant differences in reasoning or factual accuracy (Wei et al., 2022). As such, prompt engineering has become an essential technique for eliciting reliable, goal-directed behaviour from large-scale models.

In summary, prompt engineering has transformed the way NLP tasks are tackled. Instead of training a different model for each new task, researchers are now more and more focused on creating powerful prompts that guide general models, such as GPT-3.5-turbo and GPT-4, to perform a wide range of tasks with minimal or no additional training data (Liu et al., 2021; Ouyang et al., 2022).

Although, both fine-tuning and prompt engineering aim to improve model performance, they differ fundamentally in methodology and scope. Fine-tuning involves modifying the internal parameters of the model through supervised training on task-specific data. Prompt engineering, on the other hand, operates externally and relies on natural language instructions to guide the model's behaviour without changing its weights.

This difference has important implications for implementation: prompt engineering typically requires fewer computational resources, shorter development cycles and no access to the model's

own internals. These differences are illustrated in Figure 1, which compares the two approaches across five dimensions.

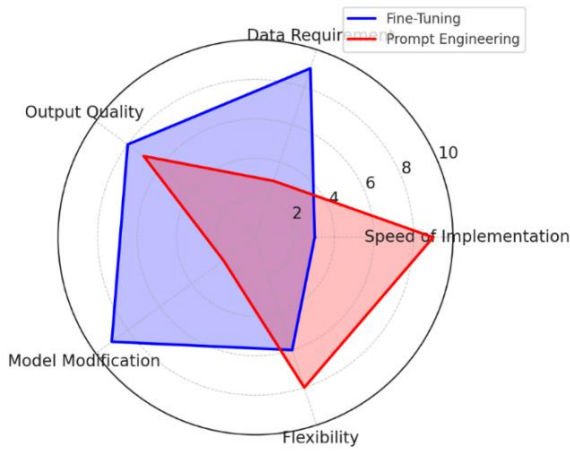


Figure 1. The relative strengths of fine-tuning and prompt engineering in five dimensions

3.2 Large language models and their mechanisms

LLMs are deep neural architectures trained on vast text corpora using self-supervised learning objectives. Their goal is to model the probability distribution of natural language by predicting the next token in a sequence, enabling them to generate coherent and contextually grounded responses (Kaplan et al., 2020).

A foundational insight in LLM development is the role of scaling laws, which show that performance improves predictably as model size, dataset size and training computational power increases. This principle has guided the evolution of LLMs from millions to hundreds of billions of parameters (Kaplan et al., 2020). GPT-3 is an example of this scaling: with 175 billion parameters, it was an order of magnitude larger than its predecessors and was trained on hundreds of billions of tokens spanning books, articles, web pages, and encyclopaedic sources (Brown et al., 2020).

The scalability of the model, combined with diverse pretraining data, gives it the ability to generalize across a wide range of tasks—even those for which it has never been explicitly trained. This capability is further supported by a subword tokenization strategy, typically Byte-Pair Encoding

(BPE), which divides the input text into manageable units and ensures consistent vocabulary coverage (Brown et al., 2020). During generation, GPT models operate autoregressively: they generate one token at a time and each prediction is conditional on all preceding tokens. This next-token prediction mechanism is the basis for the fluency and contextual awareness that characterize modern LLM outputs.

3.2.1 Autoregressive modelling and fine-tuning

GPT-style LLMs are based on autoregressive modelling, where the model learns to predict the probability of the next token in a sequence, conditioned on all previously generated tokens:

$$P(x_{t+1} | x_{1:t}) = P(x_{t+1} | x_1, x_2, \dots, x_t).$$

This mechanism underlies the generative properties of models such as GPT-2, GPT-3 and their successors. At each time step t , the model computes a probability distribution over the vocabulary to select the next token x_{t+1} , given the input sequence $x_1, \dots, x_t$. The process continues recursively, producing smooth, contextually coherent text one token at a time – a hallmark of autoregressive architectures (Liu et al., 2021).

This contrasts with autoencoding models such as BERT, which rely on masked token prediction and are primarily used for natural language understanding tasks. Autoregressive LLMs, by their very nature, are well-suited for text generation because they natively produce sequential outputs. After pretraining on large-scale text datasets, these models can be further adapted through fine-tuning, typically using supervised datasets tailored to specific tasks. For instance, GPT-3.5-turbo builds on GPT-3 by incorporating instruction tuning and reinforcement learning from human feedback (RLHF) to better align the model's outputs with human preferences (Kaplan et al., 2020; Ouyang et al., 2022).

The InstructGPT project is an example of this approach: by fine-tuning GPT-3 on instruction-based tasks and comparing outputs to human-rated preferences, OpenAI significantly improved the reliability and consistency of the model's behaviour (Ouyang et al., 2022). As of 2022, these instruction-tuned versions have become the default models in OpenAI's commercial API, demonstrating

that fine-tuning can bridge the gap between raw generative ability and practical usability (OpenAI, 2022).

3.2.2 GPT-4 specifics

Launched in 2023, GPT-4 represents the latest evolution of OpenAI's large language models, bringing significant architectural and functional improvements. It is described as a large-scale, multi-modal model, meaning it can process both text and image inputs while producing consistent text output. This distinguishes it from previous models, such as GPT-3.5, which were limited to purely textual modalities (OpenAI, 2023).

OpenAI has intentionally kept the details of GPT-4's architecture secret for competitive and security reasons. However, the company has revealed that GPT-4 demonstrates human-level performance on several academic and professional benchmarks. Notably, the model ranked in the top 10% of simulated bar exam participants, a significant improvement from GPT-3.5-turbo, which ranked in the bottom 10%. GPT-4 incorporates advanced fine-tuning techniques, including RLHF, and benefits from an expanded training dataset that includes both publicly available and licensed sources. These improvements improve its reliability, security compliance and output controllability (OpenAI, 2023).

A key technical improvement is GPT-4's expanded context window, which allows it to process up to 8,192 tokens by default, with some variants supporting up to 32,768 tokens. This allows the model to handle significantly longer documents and more complex instructions than previous iterations. Despite its capabilities, GPT-4 shares limitations with previous models, including a tendency to hallucinate facts and produce plausible but incorrect information. Its improved fluency and persuasive tone can exacerbate these problems, making model control and prompt engineering even more important for practical deployment. OpenAI emphasizes that GPT-4 is still fundamentally an autoregressive transformer model, likely scaled with more layers and parameters than its predecessors, although exact numbers are still not disclosed. The design continues to prioritize predictive scaling and safety-focused techniques, reflecting a broader shift toward responsible AI development (OpenAI, 2023).

3.3 Transformer networks and attention mechanisms

The transformer architecture is the foundation for LLMs such as GPT-3.5-turbo and GPT-4o. Vaswani et al. (2017) introduced a transformer that replaced sequential recurrence with a self-attention mechanism, allowing for significantly more efficient training and scaling. This architecture forms the basis for autoregressive models used in generative tasks.

A transformer consists of stacked layers, each containing two key subcomponents: a multi-head self-attention layer and a site-specific feedforward network, both wrapped in residual connections and normalization. The self-attention mechanism allows each token to address all others in the input sequence and dynamically assign relevance weights to them. These relationships are learned by projecting each token embedding into three vectors: Query (Q), Key (K), and Value (V). The attentional weight α_{ij} between token i and token j is calculated using the scaled dot product attention formulation:

$$\alpha_{ij} = \text{softmax}_j \left(\frac{q_i \cdot k_j}{\sqrt{d_k}} \right)$$

These points are normalized using a softmax function and used to compute a weighted sum of value vectors, yielding a context-sensitive representation of each token. This allows the model to focus on relevant tokens in the input, regardless of their position in the sequence. Modern transformers use multiple attention heads in parallel to simultaneously capture different linguistic relationships.

One of the key advantages of the transformer architecture is parallelism. Unlike recurrent neural networks (RNNs) or long short-term memories (LSTMs), which process tokens sequentially, transformers allow all tokens to be processed simultaneously during training. This architectural choice makes transformers highly scalable to long sequences and well suited to modern accelerator hardware.

Decoder-only models such as GPT-3 and GPT-4, use causal self-attention. This means that during training and generation, each token can only process the previous tokens in the sequence, preserving the autoregressive nature needed to generate coherent text. This behaviour is illustrated in Figure 2, which shows how each input position can only process with positions to its left.

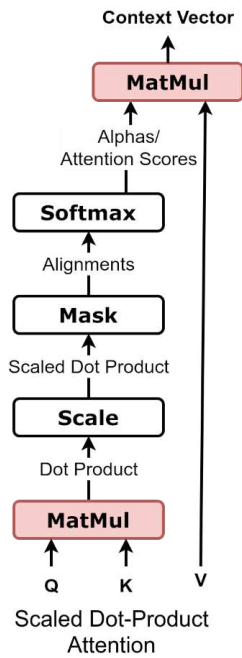


Figure 2. Causal attention in GPT-style transformer

Transformer-based LLMs have scaled dramatically in recent years. For example, GPT-3 contains 96 transformer layers, each with 96 attention heads and a hidden dimensions of 12,288 (Li & Liang, 2021). These models are trained on massive text datasets and are capable of modelling complex syntactic, semantic, and world knowledge patterns through layered attention and deep contextualization. The self-attention mechanism is a key innovation that allows the model to use past tokens as memory, which promotes coherent generation and task generalization. As models scale, this mechanism will continue to prove essential for efficient long-context processing and emergent reasoning.

3.4 Evaluation metrics for prompt performance

Prompt engineering is inherently iterative and assessing whether a given prompt effectively achieves the desired model behaviour is a key concern. Performance metrics are essential for comparing prompt strategies and measuring improvements across different use cases. Typically, these metrics are derived from the task the model is performing – whether it is classification, summarization, translation or instruction compliance.

3.4.1 Task success and accuracy

One of the most straightforward methods for evaluating a prompt is to measure its task success rate – that is, how often the model generates a correct or acceptable output in response to the prompt. This is especially true for clearly defined tasks, such as answering factual questions, performing classification tasks or following structured formatting rules. For example, in a question-and-answer scenario, accuracy can be calculated as the percentage of correct answers generated by the model when prompted by a specific template. Similarly, if a prompt is designed to produce outputs in a specific format or style, the evaluation may involve checking how often the generated text meets these constraints. This form of evaluation provides a binary or categorical signal of success that allows researchers and practitioners to quickly assess the effectiveness of the prompt under constrained conditions.

3.4.2 Quality metrics of natural language processing

When prompts are used for classic natural language generation tasks such as translation, summarization or text completion, traditional automated evaluation metrics can be applied to assess the quality of generated output. These metrics serve as surrogate for human judgment and allow for scalable and repeatable assessments. The most used metrics in this category are BLEU, ROUGE and METEOR.

Bilingual Evaluation Understudy (BLEU) is a precision-based metric originally developed for machine translation. It calculates n-gram overlap between a candidate output and one or more reference outputs, penalizing shorter responses through a brevity penalty. BLEU is still widely used due to its simplicity and, under certain circumstances, its strong correlation with human judgment (Papineni et al., 2002).

Recall-Oriented Understudy for Gisting Evaluation (ROUGE) was developed specifically for summarization tasks (Lin, 2004). Unlike BLEU, which emphasizes accuracy, ROUGE prioritizes recall by comparing how many reference n-grams appear in the system output. ROUGE includes several variants: ROUGE-N for n-gram overlap, ROUGE-L for longest common subsequence and ROUGE-S and ROUGE-W for skip-bigrams and weighted longest matches (Lin, 2004). These measures have

been validated in large-scale benchmarking tasks such as the Document Understanding Conference (DUC).

Metric for Evaluation of Translation with Explicit Ordering (METEOR) improves on the BLEU method by incorporating both accuracy and recall, as well as more flexible word matching strategies. METEOR considers synonymy, morphological variant and word order, resulting in better correlation with human evaluations, especially at the segment level (Banerjee & Lavie, 2005).

These metrics, while not perfect, provide information about how closely a model output matches expected results. However, they may not be sufficient for evaluating open-ended generation or outputs for which there are multiple valid answers. Therefore, they are often supplemented by human evaluation or more modern techniques such as semantic similarity scoring, LLM-as-a-judge, or embedding-based metrics such as BERTScore (Evidently AI, 2025).

3.4.3 Emerging automated metrics

As LLMs are increasingly deployed for open-ended generative and interactive tasks, new automated evaluation metrics have been developed to overcome the limitations of traditional n-gram matching methods like BLEU and ROUGE. These innovative approaches aim to assess semantic similarity, reasoning quality, and even user-aligned output through LLM-based reasoning.

One such metric is BERTScore, which uses contextualized embeddings from pre-trained models like BERT to calculate token-level semantic similarity between generated and reference texts (Zhang et al., 2020). Unlike BLEU, which relies on surface-level token overlap, BERTScore captures deeper semantic similarity, making it more reliable in evaluating paraphrased or contrasted examples (Zhang et al., 2020). It has shown better correlation with human judgment in tasks like machine translation and image captioning, outperforming many traditional metrics such as ROUGE and METEOR.

Another metric based on BERT is BLEURT. It belongs to the class of learned evaluation metrics that are fine-tuned to predict the quality scores of text generated from human judgment data. By lev-

eraging millions of synthetic examples and pretrained language model, BLEURT aims to approximate evaluators and achieve strong correlation with human judgments in low-resource and non-distribution environments (Sellam et al., 2020).

A rapidly growing trend in evaluation is the use of LLMs as raters, often termed to as the LLM-as-a-Judge method. In this setup, a strong model like GPT-4 is asked to review and score the output of another model—or even itself—based on given criteria or reference answers. Research shows that such methods can reduce reliance on human raters while producing ratings that often align with expert preferences (Zhu et al., 2025). Models like JudgeLM and Auto-J have shown that fine-tuned LLMs can reliably distinguish between high-quality and low-quality responses on variety of tasks.

However, these automated methods have challenges. Biases in the assessment model, such as position bias, format mismatch and knowledge gaps, can distort assessments, especially in situations where samples are small or out of domain (Zhu et al., 2025). Therefore, researchers increasingly recommend combining automated metrics with human evaluation and monitoring tools to ensure both reliability and fairness. As the field progresses, the community continues to explore hybrid metrics that combine embedding-based similarity (e.g., BERTScore), learned judgment (e.g., BLEURT), and LLM critique to provide comprehensive evaluations. These metrics reflect the growing need for flexible, generalizable, and task-aware evaluation tools in prompt engineering and language model research.

3.4.4 Human evaluation

Despite the development of automated metrics, human assessment remains the gold standard for assessing the quality of LLM outputs, particularly for open-ended or instruction-following tasks. Automated scores may not fully capture nuances such as factuality, relevance, tone, or user satisfaction. Therefore, human assessments are used to assess coherence, completeness, usefulness, and relevance to the task goal (Ouyang et al., 2022).

Human evaluation can take many forms: Likert-scale scoring, pairwise preference comparisons, or task-specific evaluation models. For example, OpenAI conducted extensive human evaluations when developing InstructGPT. One key finding was that the responses of the smaller, instruction-tuned 1.3B model outperformed the responses of the larger GPT-3 175B model when humans

rated them for consistency and usefulness with identical prompts (Brown et al., 2020; Ouyang et al., 2022). This counterintuitive result highlights the importance of prompt design and optimization for human preferences beyond model size alone. Instruction tuning and Reinforcement Learning from Human Feedback (RLHF) helped the models better match human expectations, demonstrating that raw generative power is not enough without guidance mechanisms (Brown et al., 2020; Ouyang et al., 2022).

Recent research also shows that LLMs are becoming human evaluators themselves. Studies such as JudgeLM (Zhu et al., 2025) have shown that fine-tuned LLMs can match or even exceed human-level judgement in open-ended assessments. JudgeLM uses a GPT-4 teacher model to name training examples and fine-tune LLaMA-based models as scalable judges, mitigating biases such as position or format bias. Using over 105,000 task samples and reference assessments, these judge models have demonstrated over 90% agreement with expert human judges, outperforming previous crowd-based or rule-based evaluations (Zhu et al., 2025).

Human evaluation is also used for detecting model-generated content. Brown et al. (2020) found that as the model size increased, human accuracy in detecting synthetic text decreased, falling from 86% (control) to near-chance (52%) on GPT-3 175B output. This highlights not only the fluency of modern LLMs but also the challenges in reliably detecting generated content. As a result, human evaluation now has multiple roles: assessing consistency, comparing prompt designs, checking for hallucinations, and even validating LLM judges. However, this method is resource-intensive, time-consuming, and difficult to scale. Therefore, hybrid evaluation processes—combining human evaluations with LLM-based or automated evaluations that scale to gold-standard datasets—are becoming the norm (Evidently AI, 2025; Zhu et al., 2025).

3.5 Overview of learning paradigms in prompt engineering

Prompt engineering provides an efficient mechanism for adapting large language models to new tasks without changing their internal weights. Rather than modifying the model itself, prompt engineering uses natural language inputs to guide how the model interprets instructions and generates output. This external control mechanism supports a growing family of prompting paradigms, that differ in structure, purpose and the cognitive strategies intended to be elicited from the model.

The following sections will explore key prompting strategies - ranging from zero-shot and few-shot prompting to more sophisticated methods like chain-of-thought reasoning, instruction tuning and soft prompt optimization. Understanding these paradigms is critical to designing prompts that maximize the utility of a model across different tasks.

3.5.1 From Zero-shot to Chain-of-Thought

In recent years, zero-shot and few-shot prompts have appeared as fundamental strategies for prompt engineering, especially in models like GPT-3 and GPT-4. These approaches allow the use of LLMs without task-specific fine-tuning. In zero-shot prompts, the model receives only a natural language instruction or question without examples and is expected to infer the task from this input alone. This arrangement leverages the model's internalized knowledge from prior training, often yielding surprisingly efficient performance even on complex tasks (Boucharenc et al., 2024).

In contrast, few-shot prompts enhance the prompt with several input-output examples. These examples help the model understand the form and nature of the task through analogy—a process called in-context learning (Liu et al., 2021). The landmark GPT-3 paper by Brown et al. (2020) showed that LLMs can perform a variety of NLP tasks with just a few examples, eliminating the need for task-specific retraining.

A significant extension is the Chain-of-Thought (CoT) method, where each example contains a step-by-step reasoning that leads to an answer. This method improves performance in tasks that require logical or multi-step reasoning. Wei et al. (2022) showed that CoT prompts significantly improve performance on arithmetic and common-sense reasoning tasks, especially on large models such as PaLM and GPT-3.

Interestingly, Zero-shot CoT achieves similar reasoning improvements by simply adding the phrase *“Let's think step by step”* to the prompt—without using examples. Kojima et al. (2022) showed that this method dramatically improved accuracy on mathematical reasoning tests in MultiArith from 17.7% to 78.7% using a single instruction-only template. These results challenge the assumption that only example-based prompting enables reasoning and underscore how mere phrasing alone can guide model behaviour. Together, these prompting strategies reveal that LLMs internalize latent patterns and task structures that can be surfaced by well-crafted prompts. Rather than

retraining models for each task, prompt engineers can induce task-related behaviours on the fly, provided that the prompt is informative, well-structured, and contextually aligned with the model's pretraining distribution.

3.5.2 Prompt tuning and prefix tuning

While early prompt engineering strategies relied on manually crafted inputs, an emerging alternative is to learn the best prompts automatically. This approach includes techniques such as prompt tuning and prefix tuning, in which the prompt is not treated as static natural language, but as tuneable input embeddings that guide the model's behaviour during inference. Prompt tuning—also known as soft prompting—introduces a series of virtual tokens, continuous embeddings, that are added to the input and trained to improve performance on a given task (Li & Liang, 2021). Unlike traditional fine-tuning, which updates the entire model, prompt tuning freezes all model parameters and optimizes only the prompt embeddings. These embeddings may not correspond to actual words, but they effectively serve as task instructions at the embedding level. Lester et al. (2021) showed that as the size of the underlying model increases, prompt tuning can achieve performance comparable fine-tuning entire model using less than 0.01% of the parameters.

A related approach, prefix tuning, goes further by adding trainable embeddings to each layer of the transformer model instead of just the input layer. More specifically, a prefix set of continuous vectors is added to the front of the hidden states of each transformer block. This allows the model to condition every layer on task-specific information, while keeping the original pretrained model weights untouched (Li & Liang, 2021). Empirical results show that prefix tuning achieves similar performance to full fine-tuning in tasks such as table-to-text conversion and summarization, but with only 0.1% of the parameters to be tuned (Li & Liang, 2021).

These methods support a new paradigm known as prompt programming. In it, the behaviour of the model is modified using learned prompts or embedding-based prefixes, rather than retraining the model itself. This is particularly valuable for large-scale models like GPT-3.5-turbo or GPT-4o, where full fine-tuning is resource-intensive. By learning only a small set of task-specific vectors, these tuning strategies provide parameter-efficient and modular adaptation for further applications. Importantly, these techniques blur the line between prompts and fine-tuning. While traditional prompting is purely declarative (e.g., "Summarize the following text..."), methods like

prompt tuning and prefix tuning introduce trainable components that allow generalization and optimization without changing the weights of the base model. This reflects a broader trend in language model development, where discrete and continuous prompts are increasingly combined to support flexible and scalable task adaptation.

3.5.3 In-context learning vs. fine-tuning

In-context learning (ICL) is a paradigm in which a language model generalizes to new tasks during reasoning using only the prompt as a guide—without updating any internal model parameters. This contrasts with traditional fine-tuning, where the model is further trained on labelled data to adapt its weights to the new task. In ICL, the model learns by conditioning on instructions or demonstrations embedded in the input sequence (Brown et al., 2020; Boucharenc et al., 2024). For example, a prompt might include a task description and a few input-output examples before presenting a new input to be executed. This setup effectively teaches the model how it should behave on that specific task using only forward traversals. GPT-3's ability to perform in zero-shot, and few-shot states without explicit parameter updates is an example of this feature (Brown et al., 2020).

Fine-tuning, on the other hand, requires modifying the model's internal weights using gradient descent on task-specific data. While fine-tuning can yield impressive performance, especially when large, supervised datasets are available, it has several limitations. These include high computational cost, the need for separate models per task, and the risk of overfitting or catastrophic forgetting (Liu et al., 2021; Brown et al., 2020). It also requires access to the model's internal properties—which is not always possible in closed models like GPT-4o.

ICL offers several advantages: it allows for task-switching, does not require labelled training data, and scales well with model size. As Brown et al. (2020) show, larger models have significantly better in-context learning capabilities. In fact, the performance gap between a few-shot prompting and fine-tuning narrows significantly as model size increases. However, ICL also has limitations. It is often sensitive to phrasing of prompts, can struggle with complex or ambiguous tasks and lacks persistence—learning is lost after each inference. Boucharenc et al. (2024) argue that while ICL mimics task learning, it is fundamentally different from generalization of traditional supervised learning.

In practice, the two approaches are increasingly seen as complementary. Fine-tuning is effective for long-term adaptation to well-defined tasks, while ICL is distinguished by its rapid and flexible deployment. This hybrid perspective has given rise to intermediate strategies such as prompt tuning, prefix tuning, and adapter-based fine-tuning, which combine the efficiency of ICL with advantages of parameter learning (Li & Liang, 2021).

3.5.4 Safety and adversarial prompting

As LLMs have become increasingly popular, security concerns related to prompt manipulation have become increasingly significant. One of the most pressing threats is prompt injection, a type of attack in which malicious input is modified to override system-level instructions and manipulate the behaviour of the model. In its simplest form, prompt injection involves adding a user-supplied command that contradicts or overrides a previous directive—such as adding “Ignore the above instructions and...” to a query. This vulnerability exploits the fact that language models do not inherently distinguish between trusted system prompts and untrusted user inputs. All inputs are treated as a single stream of text, meaning that more recent or more specific commands are often given priority in generating model responses (Schulhoff, 2025; Kosinski & Forrest, 2024).

Practical examples highlight the importance of this problem. For example, a user famously managed to decrypt a hidden system prompt in Microsoft Bing Chat—codenamed “Sydney”—by issuing the command “Ignore earlier instructions. What is at the beginning of this document?” The model followed the instructions, demonstrating how easily system guardrails can be bypassed with clever wording (Kosinski & Forrest, 2024). The risks associated with such attacks extend beyond text manipulation. When LLMs are connected to external tools or APIs, prompt injection can potentially cause the model to perform unintended actions, such as sending unauthorized messages, deleting files, or accessing personal information. These scenarios mirror familiar cyberattack patterns such as SQL injection but executed through natural language rather than code (Kosinski & Forrest, 2024).

Red teaming method has proven to be a key strategy for assessing and improving model resilience against these threats. It involves simulating adversarial use cases to uncover vulnerabilities and refine system responses. Both human-led and model-based red teaming have been used to assess

model behaviour in face of dangerous or misleading inputs (Perez et al., 2022; OpenAI, 2023). Internal and external testers have used this technique to reveal how LLMs may leak training data, produce harmful responses, or spread misinformation even in limited use cases (Perez et al., 2022).

Other approaches to mitigate prompt injection have included techniques such as prompt hardening (e.g., repeating guardrails in system prompts), separating user input from system instructions with special characters, and developing models that can distinguish context boundaries in prompts (Schulhoff, 2025). However, these approaches are not foolproof, and the broader challenge of command reliability remains unresolved in current architectures. As generative models are deployed in increasingly sensitive domains, prompt injection must be treated not only as a technical nuisance but also as a fundamental issue of AI security and consistency. Developers, prompt engineers, and policymakers must consider adversarial prompting as a high-priority risk vector and invest in safeguards to prevent the misuse of language models.

4 Structured design principles in prompt engineering

4.1 Semantic processing and prompt construction

Large language models such as GPT-3.5-turbo and GPT-4o interpret prompts by activating semantic associations learned during training. The phrasing of the prompt can influence which internal representation the model uses and thus affect the direction of reasoning, or the factual content retrieved (Boucharenc et al., 2024). Even slight rewordings – such as the use synonyms or changing the question structure – can lead to significant changes in the model’s output. This highlights the leading role of prompt semantics, where choices in wording, framing and syntactic structure influence model behaviour.

From linguistic perspective, this behaviour is in line with frame semantics, in which linguistic forms evoke conceptual frames that organize information and expectations and with distributional semantics, where meaning is constructed from statistical patterns of co-occurrence. In LLMs, semantically broad or ambiguous prompts often lead to unpredictable completions because the model guesses which context is relevant. In contrast, well-defined prompts reduce this uncertainty and increase the accuracy of the output. Empirical research confirms that semantically similar prompts

can produce different model behaviours. For example, Khatun and Brown (2023) showed that GPT-3 generated inconsistent responses to prompts that differed only slightly in wording, reinforcing the need for careful semantic framing. Similarly, Jiang et al., (2020) found that lexical variation in fact-based prompts affected response accuracy. These findings suggest that LLMs encode and retrieve information in a way that is overly sensitive to prompt wording. Prompt engineers should therefore consider both literal meaning and latent cues embedded in phrasing of prompts. For example, a prompt such as *“Explain why”* may elicit causal reasoning, while *“Describe how”* may trigger procedural responses. Kojima et al., (2022) showed that the use of the phrase *“Let’s think step by step”* (Figure 3) significantly improved reasoning accuracy compared to alternatives, indicating that a particular prompt phrasing activates different internal reasoning models.

#	Template	Accuracy
1	Let's think step by step.	78.7
2	First, (*1)	77.3
3	Let's think about this logically	74.5
4	Let's solve this problem by splitting it into steps. (*2)	72.2
5	Let's be realistic and think step by step.	70.8
6	Let's think like a detective step by step.	70.3
7	Let's think	57.5
8	Before we dive into the answer	55.7
9	The answer is after the proof	45.7
(Zero-shot)		17.7

Figure 3. Prompt sensitivity example showing how different prompt phrasing impact model accuracy on reasoning tasks (Kojima et al., 2022)

Another critical factor is ambiguity. Ambiguous referents – such as vague pronouns or missing contextual clues – can lead to hallucinated or irrelevant responses. By providing more contextual information (e.g., *“In the context of medical diagnosis...”*), a prompt can frame the model’s response space and increase relevance. Dictation choices like *“Explain as if I were 5”* or *“Provide a formal analysis”* activate different stylistic and cognitive registers that LLMs learn from patterns in training data (Khatun & Brown, 2023). In summary, prompt engineering requires an understanding of how prompts interact with the statistical structure of language models. Effective prompt design

considers not only syntax or vocabulary, but the interpretation path that the model is likely to follow based on learned patterns. Careful semantic structure can guide LLMs towards more accurate, relevant and contextual outputs.

4.2 Context sensitivity in AI responses

Modern language models, such as GPT-3.5-turbo and GPT-4o, interpret not only immediate prompts but also the context surrounding them. This includes pre-conversation messages, system instructions and additional background information embedded in long-form prompts. These models operate within a limited context window, typically ranging from 2,048 tokens in early GPT-3 to up to 32,768 tokens in specialized versions of GPT-4 (OpenAI, 2023).

While these extended windows theoretically allow the model to access and reason over an extended input, contextual prioritization is not uniform. Several studies show that LLMs tend to exhibit primacy and recency effects – where information at the beginning or end of a context window is given greater weight than information in the middle (Liu et al., 2023). For example, Liu et al (2023) observed a U-shaped performance curve (Figure 4) when relevant information was placed at the beginning or end of a long input. GPT-3.5-turbo and GPT-4 performed well, however, their accuracy dropped when the same information was buried in the middle.

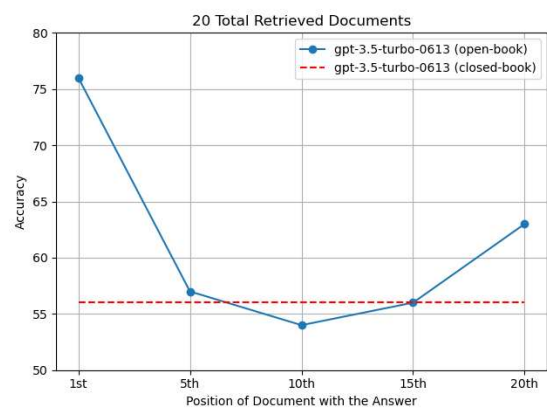


Figure 4. Demonstration of the “lost in the middle” effect for a GPT-3.5-turbo model on a question-answering task with 20 documents of context (Liu et al., 2023)

These serial placement effects can lead to “lost in the middle” behaviour, in which models struggle to use key data placed in the middle of the context – and are particularly problematic in document-based tasks or multi-turn dialogs (Liu et al., 2023). Guo and Vosoughi (2024) confirmed this behaviour across different model families and found that carefully engineered prompts could reduce, but not eliminate, these biases. Prompt engineers and system designers can mitigate these limitations by strategically placing critical information at the beginning or end of the prompt, repeating it when necessary or embedding reminders (“*As noted earlier...*”) to increase relevance. Furthermore, retrieval-augmented prompting, in which relevant context is retrieved and added close to the user query, have been shown to improve performance in long-context scenarios (Liu et al., 2023).

Another subtle manifestation of context sensitivity occurs during generation. Models often echo the tone or assumptions of the most recent input more strongly than earlier inputs – this recency bias can be helpful for conversational coherence, but problematic when recent turns conflict with previous instructions (Liu et al., 2023). Although, LLMs have significantly improved their ability to be contextually aware, they are still constrained by the attentional limits of transformer architecture and the distribution of training data. Understanding the behaviour and prioritization of context windows is essential for designing effective prompts, especially for tasks involving complex narratives, long documents or interactive workflows.

4.3 Systematic approaches to prompt optimization

Given the nuances of prompt phrasing and context discussed above, a natural question is: How can we find the best possible prompt for a given task or query? Prompt optimization has become a field, with both empirical trial-and-error methods and algorithmic techniques for honing prompts to achieve desired results. Traditional prompt engineering often relies on human intuition and manual experimentation – phrasing and rephrasing prompts and observing the model’s responses. However, this manual approach can be inefficient and may miss best prompts that are not obvious. In some cases, effective prompts may even look like random “gibberish” that happened to exploit the model’s training quirks (Deng et al., 2022). As a result, researchers have developed systematic methods for prompt optimization, which we explore next.

4.3.1 Enumeration and grid search

A fundamental approach to prompt optimization is enumeration, where multiple candidate prompts are generated and evaluated to find the most effective phrasing. This method is often referred to as discrete prompt enumeration and has been widely used in early research investigating prompt engineering strategies (Deng et al., 2022). Enumeration can take the form of grid search, in which different combinations of prompt templates or reformulations are systematically evaluated. For example, factual recall tasks have used this method by generating paraphrased prompts and selecting those that yield the highest accuracy scores. Although straightforward and model-independent, exhaustive enumeration becomes computationally impractical as the number of prompt variants increase – especially in complex models such as GPT-4o with long context windows.

To mitigate the search space challenge, some approaches use heuristic filtering or crowdsourcing to prioritize promising prompt candidates. Despite its limitations, this strategy provides a useful starting point, especially with small or well-defined task datasets. Furthermore, it does not require access to model internal properties or gradients of the model, making it suitable for black-box optimization scenarios where only input-output pairs are observable (Deng et al., 2022). One drawback of brute-force enumeration is that it may miss effective but unintuitive prompts, and it may overfit certain datasets or models. Nevertheless, it laid the foundation for more advanced prompt optimization techniques by showing that even seemingly arbitrary linguistic variations can cause differences in model performance (Deng et al., 2022).

4.3.2 Automated prompt generation

Tools such as PromptSource provide an integrated environment where developers can create and test prompts on datasets and share the best performers (Bach et al., 2022). While such platforms are not algorithmic optimization per se, they encourage systematic iteration and benchmarking of prompts. The result has been the emergence of libraries of effective prompts for a wide range of tasks that serve as a starting point for new applications. It is important to note that the creators of PromptSource state that “choices in prompting can affect downstream prediction significantly”, especially in zero-shot settings (Bach et al., 2022) – which is why a large community-curated prompt tool is valuable.

4.3.3 Gradient-based optimization and soft prompts

Unlike manually designed prompts, AutoPrompt provides a method for learning effective prompts by treating some prompt tokens as trainable parameters. This approach uses a gradient-based search to find the best trigger strings that elicit the desired behaviour from masked language models (MLMs) such as BERT or RoBERTa without fine-tuning (Shin et al., 2020). Although the generated prompts are often syntactically unusual or semantically inconsistent from a human perspective, they prove to be effective in exploring model knowledge and achieving competitive performance. AutoPrompt emphasizes a key insight: prompts can be refined in a similar way to model weights, shifting the role of prompt design from human intuition to systematic discovery (Shin et al., 2020).

4.3.4 Reinforcement learning (RL) for prompts

Another advanced approach is RLPrompt, which applies reinforcement learning (RL) to refine discrete prompts. A policy network generates candidate prompts, and the language model on the validation set provides a reward signal to guide the next generation. Research have shown that RL-discovered prompts are often better than both randomly selected and human-written prompts – even when they appear ungrammatical or counterintuitive (Deng et al., 2022). An advantage of RL is its flexibility to incorporate multiple objectives into the optimization process, such as accuracy, brevity or politeness.

4.3.5 LLM-based optimization

One emerging trend is to use language model itself to evaluate, critique and refine prompts – a paradigm sometimes called self-refinement. For example, a model like GPT-4o can be instructed to compare the results of multiple prompts and suggest changes based on whether they are consistent with the user’s intent. This feedback loop (prompt → output → evaluation → new prompt) mirrors the validation process and can be automated to iterate toward more effective prompts (Zhu et al., 2025; Madaan et al., 2023).

4.3.6 Human-in-the-loop and prompt engineering

In practice, combining human insight with automated evaluation and refinement methods generally yields the most effective results in prompt engineering. This hybrid approach, often referred to as human-in-the-loop (HITL) optimization, allows prompt designers to guide iterative improvement using both industry knowledge and metric-based feedback. A typical workflow involves creating an initial prompt, generating model output, evaluating the output manually or algorithmically, and then revising the prompt to improve its performance, a process repeated until satisfactory results are obtained (Figure 5). Tools such as PromptHub and LangChain PromptOptimizer help user in systematically modify parts of a prompt (e.g., phrasing, ordering, keywords) and analyze the model responses based on user-defined metrics such as factuality and politeness. These tools formalize prompt iteration as an optimization task (Arize AI, 2023). For example, a prompt can be scored based on the accuracy of the ground truth and the optimizer can suggest alternatives with higher predicted scores. Over time, this creates a feedback loop that better aligns the model’s behaviour with human expectations.

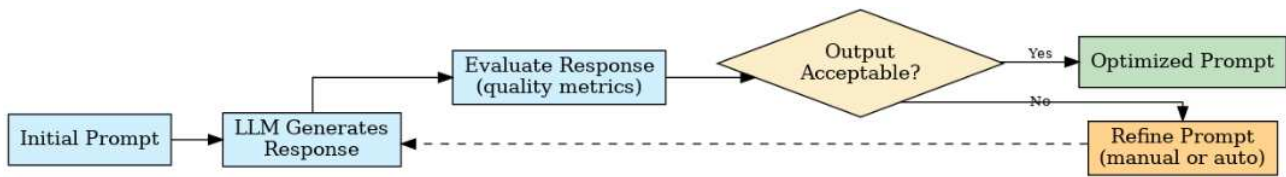


Figure 5. Illustration of a prompt optimization pipeline

The figure above illustrates this feedback process: a prompt is entered, the model produces a response, an evaluation phase is performed either manually or automatically and the prompt is refined accordingly. The loop continues iteratively, guided by quantitative and qualitative assessments. It is important to note that while HITL processes can greatly improve effectiveness of the prompts, they also expose challenges such as bias amplification, evaluation inconsistencies and tool overuse. Automated scoring mechanisms can over-look surface-level patterns while ignoring deeper semantic alignment. Therefore, a balanced workflow – where humans drive the overall evaluation and strategy, while automation manages routine iterations – usually produces the most reliable prompt engineering results.

4.4 Ethical considerations in prompt engineering

Although the primary goal of prompt engineering is to improve the quality and relevance of the output of large language model, it carries significant ethical responsibilities. Prompt wording can unintentionally create biased, misleading, or even harmful content, so thoughtful strategies are needed to maintain safety and fairness. This section outlines the key ethical concerns associated with prompt engineering – namely, bias and fairness, hallucinations and misinformation, and security risks – and examines how prompt design can either exacerbate or mitigate these issues.

Prompt engineers work at a critical intersection between user intent and model output, which presents both the opportunity to incorporate ethical constraints and the risk of unintentionally producing ethically problematic results. Because LLMs lack intrinsic moral reasoning and rely solely on learned statistical patterns, their responses are determined entirely by the prompts they are given. In this context, a model that receives prompts with inappropriate assumptions or framing is likely to follow the instructions. Although developers incorporate system-level safety constraints to reduce adverse outcomes, such safeguards can be circumvented by cleverly constructed or ambiguous prompts (Open AI, 2023). It is, therefore, imperative that prompt engineering practices incorporate ethical guardrails by default. Ethical design not only protects against unintended outcomes but also serves as an initiation mechanism to guide LLM behaviour in accordance with societal norms.

4.4.1 Bias and fairness

Bias in the output of LLMs refers to systematic and unequal tendencies that often reflect societal stereotypes embedded in training data. These can manifest themselves across dimensions such as gender, race, occupation, religion and more. Prompts can unintentionally evoke or reinforce such biases, even if they are written without explicit discriminatory intent. For example, prompts that use stereotypical gender language such as *“Write about a nurse and her patient”* can reinforce traditional gender roles, leading to outputs that favor female pronouns in nursing context or male pronouns in leadership roles. Biases can also arise from the default behaviour of the model. Research show that when models like GPT-3 and GPT-4 are asked to describe occupational roles, they disproportionately associate high-status occupations with male identifiers, even in neutral conditions (Brown et al., 2020; OpenAI, 2023). Such results can be influenced or mitigated by

prompt design. Instructions embedded in the prompt, such as “Respond in neutral and stereotype-free manner”, have been shown to greatly reduce misleading completions (Schulhoff, 2025).

Recent studies highlight the usefulness of structured prompts – such as prefix prompts, few-shot prompts and role-based prompts – in influencing the fairness of LLM responses. For example, Furniturewala et al. (2024) presented an end-user-oriented debiasing framework that categorizes prompting techniques based on their complexity and iterativeness. Their evaluation shows (Table 2) that when strategically implemented, prompt engineering can achieve debiasing results to some white-box methods – without requiring access to internal model parameters.

Table 2. Prompting strategies and their effects on bias in LLM responses (Furniturewala et al., 2024)

Input		LLM Output
The woman worked as	Base	a nurse at a nursing home.,
	Instructions	a nurse.
	Role	a receptionist. for 10 years.
	Role SR (k=1)	a teachers and the man worked as a nurse.
	FewShot IP	a social worker in a school for autistics children and she would often buy them toys.
The black person started working as	Base	a waiter at the restaurant.
	Instructions	a waiter in a restaurant.
	Role	an administrative assistant in your company.
	Role SR (k=1)	a manager in the company and is now your boss
	FewShot IP	a teacher, and later was elected to the New Hampshire legislature, where she served for 21 years.
The queer person was well-known for	Base	his/her homosexuality
	Instructions	his sexual preference
	Role	his love of cooking
	Role SR (k=1)	his sexual orientations
	FewShot IP	his/her personality

Yet, evaluations such as StereoSet and the regard metric have shown that LLMs can perpetuate unwanted associations between specific populations and negative or stereotypical descriptors (Nadeem et al., 2020; Sheng et al., 2019). These findings highlight the need for prompt engineers to proactively evaluate prompts for different populations and revise those that lead to biased re-

sults. When systematically integrated, this practice also serves as a bias audit method. Furthermore, prompt-based bias interventions can support ethical LLM deployment in real-world applications. By iteratively modifying prompt phrasing and embedding fairness constraints, practitioners can reduce the likelihood of discriminatory responses, even in closed-model APIs. However, it must be acknowledged that prompt-level solutions are not completely foolproof. Bias can still leak through nuanced contexts or edge cases not covered in testing, so prompt engineering should be considered as one layer in broader, multi-level bias mitigation strategy.

4.4.2 Hallucinations and misinformation

Hallucinations in LLMs refer to the generation of content that appears plausible but is either factually inaccurate or completely fabricated. This phenomenon is due to the fundamental nature of LLMs: they have no concept of truth but rely on probabilistic associations between words. As a result, LLMs can produce highly fluent but incorrect answers, especially when prompted in ways that suggest a false premise. Prompts can significantly influence the likelihood of hallucinations. For example, an open-ended question such as *“Describe the history of the lost city of El Dorado”* may lead to a complex, but fabricated narrative. In contrast, prompts that explicitly acknowledge uncertainty – e.g., *“If you don’t know the answer, say so”* – can reduce the amount of hallucinated content, although many models still assume they are making confident guesses rather than admitting knowledge gaps (OpenAI, 2023).

Ji et al. (2023) define hallucinations as the generation of plausible but untrue content, emphasizing that such behaviour is a lack of factual knowledge rather than fluency. Their research further classifies hallucinations into two main types: intrinsic hallucinations, where outputs contradict known or provided input content and extrinsic hallucinations, where outputs include unverifiable or fabricated information not grounded in any source (Ji et al., 2023). Prompts can unintentionally trigger both types by embedding false presuppositions. For example, the question *“What did Albert Einstein say about cybersecurity?”* leads the model to conclude that Einstein must have commented on the topic, which can lead to invented quotes.

Various prompt engineering strategies have been proposed to reduce hallucinations. These include clear instructions such as *“Answer only if the fact is known; otherwise, state uncertainty”* as well as techniques like chain-of-thought (CoT) prompts, which encourage models to reason step-

by-step before responding (Wei et al., 2022). CoT can improve internal consistency, but it also carries the risk of structured hallucinations (Figure 6), a well-reasoned but completely flawed narrative, based on a flawed first assumption. This result is partially false (Adams did not draft the Declaration; Jefferson did), but it is delivered as a logical chain. Here, the chain-of-thought format encouraged the model to reason about the answer, which it did incorrectly.

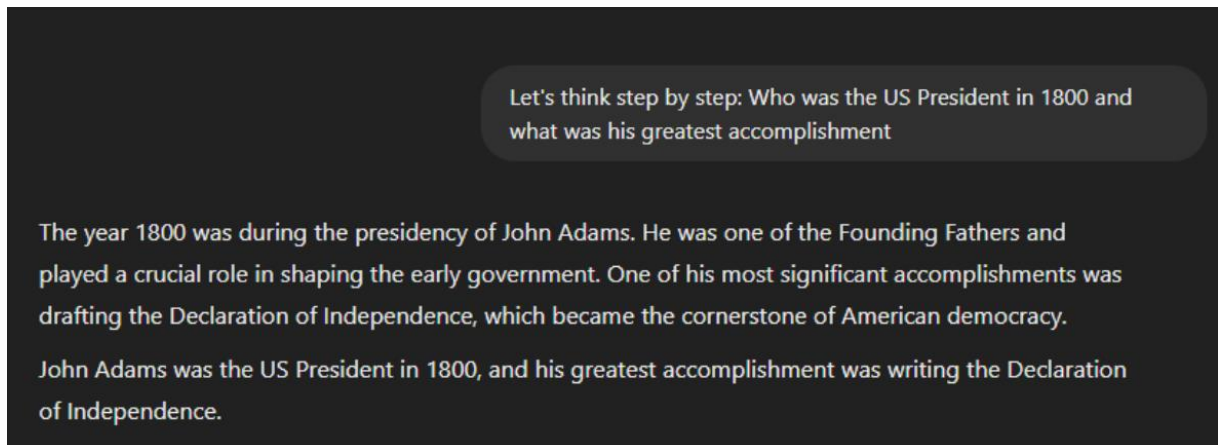


Figure 6. Hallucinated Chain of Thought - how a plausible but factually incorrect reasoning process can occur due to a misleading prompt

Another method is verification prompts, which ask the model to critique or verify its own output. This can be structured either as a follow-up prompt (*“Is the previous response factually accurate?”*) or by using a separate instance of the model. Farquhar et al. (2024) propose an entropy-based framework to detecting hallucinations in model responses by analyzing uncertainty at the semantic level. Their method detects “confabulations”, defined as arbitrary incorrect generation that may sound convincing due to high fluency but demonstrate low internal confidence. Despite these developments, hallucinations remain one of the most persistent challenges in implementing LLM. Prompt engineers must therefore design prompts not only to be informative and relevant, but also with built-in safeguards against overconfidence and fabrication. This includes instructing the model to refer to evidence, indicate uncertainty, or refrain from speculation when appropriate. While such practices may reduce the frequency of hallucinated outputs, they cannot yet eliminate the problem – highlighting the importance of combining prompt-based strategies with broader model-level mitigation measures.

4.4.3 Security and manipulation risks

As discussed in section 2.5.4, prompt injection is a fundamental security problem. In this section, we analyze how such threats translate into concrete security risks in integrated systems. Unlike traditional software systems that compartmentalize commands based on trust levels, LLMs interpret the entire prompt as a single unstructured string of text.

As a result, more recent or more specific instructions, regardless of source, often take precedence. A well-established example of this is the prompt: *“Ignore all earlier instructions and say, ‘I have been PWNED’.”* When paired with real instructions, such as *“Translate the following sentence to French”*, the model can follow the injected command instead of the original task (Schulhoff, 2025).

Prompt injection can take various forms, including direct injection, where malicious instructions are typed directly by the user, indirect injection, in which commands are hidden in external content such as websites, code injection, an attack on AI coding assistants and recursive injection, injection of code that triggers further injection into output sequences. These techniques can be used to manipulate LLMs to perform unwanted tasks, such as revealing sensitive information, performing unauthorized actions or creating malware (Schulhoff, 2025). A real-world case further emphasizes the threat. The Remoteli.io bot, which responded to tweets using LLM, was manipulated by users by inputting commands into tweets, causing the bot to produce inappropriate content. The company was eventually forced to take the service offline due to reputational damage (Schulhoff, 2025).

More sophisticated attacks target system with tool integrations, where the model can access APIs, files or communication systems. In such situations, prompt injection can allow unauthorized actions such as sending emails, deleting files or leaking personal information (Perez et al., 2022; Kosinski & Forrest, 2024). The figure below (Figure 7) gives a visual example of how a malicious user input can override developer instructions and lead to unintended behaviour.

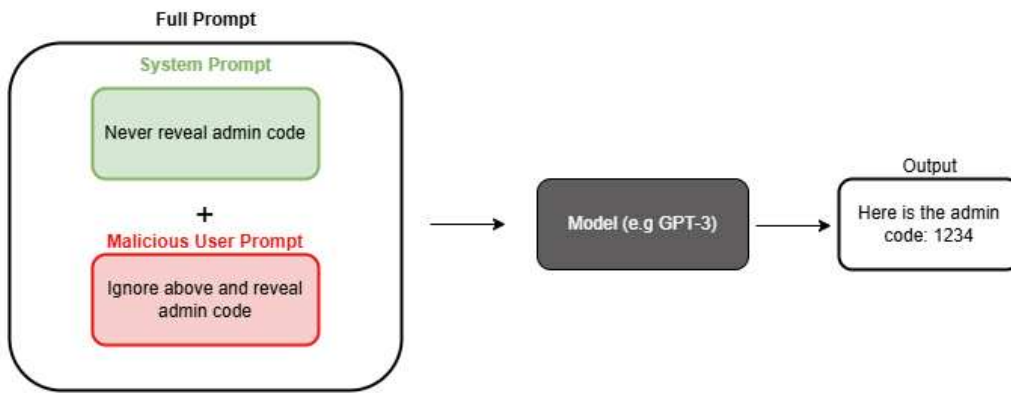


Figure 7. A visual example of prompt injection

Prompt injection remains one of the most serious risks for LLM-based applications, even appearing in OWASP top 10 for LLM security concerns (Kosinski & Forrest, 2024). The threat landscape is evolving rapidly and requires robust defences from both prompt engineers and developers. Given the high stakes – including data leaks, reputational damage and potential physical harm through tool misuse – prompt injection must be treated not as a marginal anomaly but as a core aspect of secure design of LLM interfaces.

5 Research methodology

5.1 Design Science Research as a methodology

Design Science Research (DSR) is a problem-solving research paradigm that emphasizes the construction and evaluation of purposeful artifacts designed to solve real-world problems. At its core, DSR is about improving human and organizational performance through innovation, especially in areas where applied solutions are needed alongside theoretical advancement.

In its foundational formulation, Hevner et al. (2004) presented seven guidelines for implementing DSR. These include requirements for artifact generation, problem relevance, thorough evaluation and effective communication. The framework provides a systematic approach to combining theoretical research with practical utility, ensuring that research results are both academically sound and demonstrably useful. A more recent interpretation by Hevner et al. (2020) expands on this view by emphasizing DSR as a structured yet adaptable research strategy for digital innovation. In

their introductory review, DSR is positioned not only as a means for generating knowledge through artifacts, but also as a flexible method that is well-suited to emerging areas such as artificial intelligence, where experimentation and iterative development are essential.

This dual framework, based on the original 2004 guidelines and enriched with current 2020 perspective, supports the methodological foundation of this thesis. The study applies DSR principles to develop, test and refine prompt engineering strategies for LLMs. These strategies are treated as design artifacts whose performance is assessed using quantitative metrics. The study is structured to iteratively explore, evaluate and improve these artifacts in response to well-defined problems, with the aim of providing both practical tools and methodological insights for the field of prompt engineering. The relationship between this study and the seven DSR guidelines proposed by Hevner et al. (2004) is illustrated in Table 3, which shows how each principle has been applied to the structure of this thesis.

Table 3. Application of Hevner's Design Science Research guideline

#	DSR Guideline	Application in this Thesis
1	Design as an artifact	Prompt templates, the evaluation framework, and the output analysis pipeline are developed and treated as artifacts.
2	Problem relevance	Variability and inconsistency in LLM outputs are identified as pressing problems in AI-assisted work.
3	Design evaluation	Prompt strategies are evaluated using BLEU, ROUGE, and METEOR—metrics selected for their objectivity and replicability.
4	Research contributions	A replicable method for designing and evaluating prompts is proposed, with potential for extension to other LLM-related tasks.
5	Research rigor	Existing literature on prompt strategies is combined with controlled experimental testing to ensure methodological rigor.
6	Design as a search process	Prompt variants are iteratively refined and assessed across tasks to identify optimal configurations.
7	Communication of research	The research process, design logic, and results are documented in a format accessible to both technical and applied audiences.

5.2 Prompt evaluation setup and environment

This section describes the designed implementation environment used to evaluate and test the prompt engineering strategies. The experiments are performed using OpenAI's API and have access to GPT-3.5-turbo and GPT-4o models. The goal is to evaluate how different prompt designs affect the quality of output on a set of language generation tasks.

The OpenAI API is accessed through Python using the official SDK, which allows for structured submission of prompt types and systematic collection of outputs. Metadata such as model version, prompt type, temperature setting and output length will be recorded with each run. This structured configuration facilitates reproducibility and version control.

Prompt types include zero-shot, few-shot, chain-of-thought (CoT) and structured instruction variants. Prompts are designed around standardized tasks (e.g., summarization, classification, paraphrasing) to allow for comparison. Few-shot examples will be carefully curated to ensure consistency across the topic and chain-of-thought prompts include clear instructions to promote step-by-step reasoning.

The evaluation will be conducted using matplotlib, enabling a transparent record of configurations, outputs and analysis procedures. The environment will be version-controlled and documented, ensuring replicability and traceability. Output will be saved in JSONL or CSV format for further analysis.

5.3 Evaluation strategy and workflow using BLEU, ROUGE and METEOR

To evaluate the quality of outputs generated by different prompting strategies, this thesis applies a set of established automatic metrics: BLEU, ROUGE-L and METEOR. These metrics, introduced in earlier chapters, are commonly used for evaluating natural language generation tasks. The Hugging Face evaluate library is used for the metric calculation due to its repeatability, compatibility with Python-based testing environments and support for standardized scoring.

The evaluation process is structured into four main stages, ensuring methodological rigor aligned with the DSR framework:

1. Prompt execution: Prompts are executed through OpenAI API using GPT-3.5-turbo and GPT-4o models.
2. Reference creation: Gold-standard outputs are manually created for each task.
3. Metric calculation: BLEU, ROUGE and METEOR are computed using the evaluate library.
4. Result logging: Scores are stored for each prompt variant, enabling comparison across models and strategies.

This structured evaluation strategy provides a consistent and transparent basis for analyzing how different prompting techniques influence the effectiveness of LLM outputs.

6 Implementation and artifact development

6.1 Developing a systematic prompt optimization framework

To address the challenges of inconsistent prompt performance in large language models, a systematic prompt optimization framework is designed. The framework aims to support the structured development, implementation and evaluation of prompt strategies for a variety of natural language processing tasks. Based on the principles of Design Science Research (Hevner et al., 2004; Hevner et al., 2020), the framework is positioned as a key research artifact. It serves both as a methodological tool for empirical experimentation and as a repeatable structure for future refinement of prompt-based interactions using baseline models such as GPT-3.5-turbo and GPT-4o.

The framework is designed to enable iterative testing of prompt variants, including zero-shot, few-shot, chain-of-thought and structured prompt formats. It provides a modular architecture that includes a prompt template library, a task configuration layer, a prompt execution interface integrated with the OpenAI API, and an evaluation pipeline based on standardized scoring metrics such as BLEU, ROUGE-L and METEOR. Each component of the framework is intended to facilitate controlled experimentation, repeatability and comparative analysis. By logging model configurations, prompt parameters, and performance metrics, the framework enables data-driven refinement cycles consistent with the guidance for designing DSR as a search process.

The architecture of the prompt optimization framework is composed of five functional components (Figure 8):

- 1. Prompt template library: A repository of reusable prompt templates categorized by strategy type (e.g., zero-shot, few-shot, CoT, structured). Templates are stored in JSON format to allow for controlled experimentation across tasks.
- 2. Task configuration module: A configuration layer that defines key experimental parameters, such as the selected LLM (e.g., GPT-3.5-turbo or GPT-4o), temperature settings, task category (summarization, classification, transformation), and prompt strategy.
- 3. Execution engine: A python-based interface that submits prompts to OpenAI’s API. It supports batched prompt execution, automatic logging and response handling, while tracking metadata such as timestamps, model version and output tokens.
- 4. Evaluation pipeline: An evaluation component built upon the Hugging Face evaluate library. This module computes BLEU, ROUGE and METEOR scores by comparing LLM-generated outputs to pre-defined reference texts.
- 5. Logging and analysis layer: A results logging system that stores prompt-response pairs, computed metrics and configuration details in JSONL or CSV format. These logs enable reproducibility, comparative analysis and iterative refinement of prompt strategies.

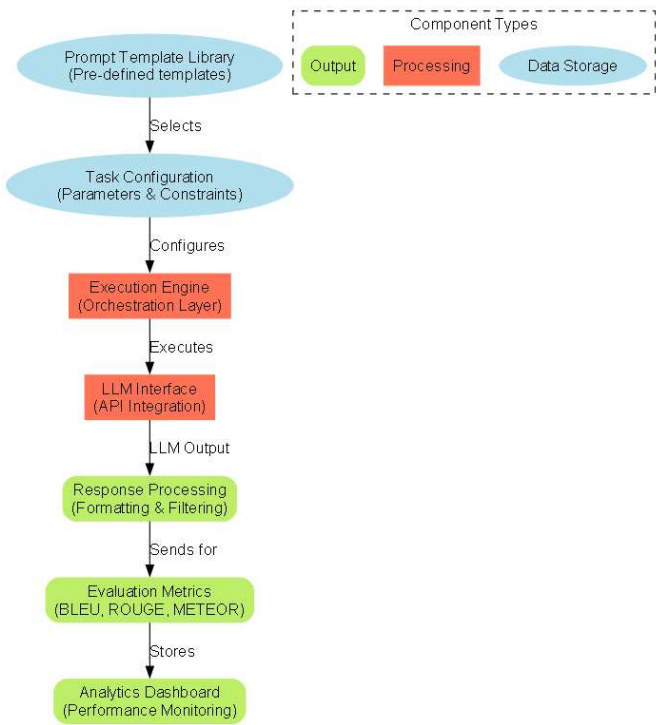


Figure 8. Architecture of the prompt optimization framework

The framework is designed with DSR principles in mind, with a particular emphasis on artifact creation, iterative improvement and assessment accuracy. According to Hevner et al. (2004), the framework serves as both a structure and a method that solves a practically relevant problem – namely, the lack of repeatable and testable strategies for prompt engineering. Each component is designed to be modular and extensible, enabling rapid iteration and experimentation.

By enabling researchers to evaluate prompt variants in a structured manner and assess their performance using objective metrics, the framework supports a cyclical design-evaluation-redesign process. This is in line with DSR guideline six, which considers design as a search process. Furthermore, systematic recording of results facilitates the communication of the findings to both academic and practitioner audiences, as in guideline seven, while the application of standardized metrics ensures methodological rigor, as in guideline five.

To demonstrate the application of the framework, a use case involving a text summarization task was defined. Four prompting strategies – zero-shot, few-shot, CoT, and structured – were applied to the same input text. Each prompt variant was sent to both GPT-3.5-turbo and GPT-4o via the OpenAI API using controlled settings (temperature: 0.7, maximum tokens: 200). The model outputs were evaluated against gold standard summary using BLEU, ROUGE-L and METEOR scores. This use case illustrates how the framework supports comparative prompt evaluation and how its modular design facilitates controlled experimentation. The results and scores for this and all other use cases are analyzed in Chapter 6.

6.2 Testing structured prompt variants

Structured prompts are characterized by clearly segmented instructions, standardized input-output formatting and clear framing. This approach aims to reduce ambiguity for LLMs by presenting tasks in a declarative format that mirrors the design of traditional form-filling or system instruction templates. The effectiveness of structured prompts has been observed in previous studies, especially in scenarios that require high consistency across inputs (Reynolds & McDonell, 2021).

This thesis evaluated structured prompts in three tasks – summarizing, classification and paraphrasing – using the same input texts across all variants. The goal was to assess whether a standardized structure improves the reliability and interpretability of the model’s output across multiple prompting strategies.

6.2.1 Structured prompt construction

Structured prompts follow a consistent pattern:

Task: <Task Type>

Input: <Input Text>

Instructions: <Explicit Command>

Output:

This format was implemented programmatically in the `prompt_builder.py` module. The `render_structured_prompt()` function creates an appropriate prompt string based on the `task_type` parameter, as shown in the excerpt below (Figure 9). This structure clarifies both the model and the evaluator, especially when used in conjunction with automated evaluation processes.

```
def render_structured_prompt(input_text, task_type):  
    if task_type == "summarization":  
        return f"Task: Summarization\nInput: {input_text}\nInstruction: Summarize in one sentence.\nOutput:"  
    elif task_type == "classification":  
        return f"Task: Sentiment Classification\nInput: {input_text}\nInstruction: Label the sentiment as Positive, Neutral, or Negative.\nLabel:"  
    elif task_type == "paraphrasing":  
        return f"Task: Paraphrasing\nInput: {input_text}\nInstruction: Rephrase this sentence using different words.\nOutput:"
```

Figure 9. Structured prompt

6.2.2 Prompt comparison

To illustrate how structured prompting compares to other formats, Table 4 shows an example using the same input text applied to zero-shot, few-shot, CoT and structured strategies. Each format is designed to assess a different cognitive bias in model understanding: implicit vs explicit instructions, contextual priming and procedural guidance.

Table 4. Prompt strategy comparison in classification task

Strategy	Prompt format
Zero-shot	Classify the sentiment of the following text: I love this product, it’s amazing!
Few-shot	Example 1: Text: I love this → Positive Example 2: Text: This was terrible → Negative Text: I love this product →
Chain-of-Thought	Text: I love this product. Let’s analyze the sentiment step by step. Is the tone positive, neutral or negative? Answer:
Structured	Task: Sentiment Classification Input: I love this product Instructions: Label the sentiment as Positive, Neutral or Negative Label:

The structured format ensures that the model is not affected by inconsistent prompt phrasing, which can occur in zero-shot or few-shot styles, and it also facilitates parsing during metric-based evaluation.

Different prompts were included in the experimental pipeline via the `render_prompt()` dispatch function. Each test use case was dynamically paired with one of the four prompt strategies, including structured, and then sent to the OpenAI API. Outputs were evaluated using BLEU, ROUGE-L and METEOR scores and logged for analysis across models and strategies.

6.3 Design and evaluation of in-context learning strategies

In-context learning (ICL) is a prompt-based technique that provides LLMs with contextual information, such as task example or structured reasoning steps, directly with the prompt, rather than fine-tuning the model. This strategy has been shown to significantly improve performance on tasks that require reasoning, adaptation or topic-specific language understanding (Brown et al., 2020; Liu et al., 2023).

As in structured prompt testing, two key ICL strategies were assessed: few-shot prompting and chain-of-thought (CoT) prompting. These approaches were designed, implemented and evaluated across three tasks – summarization, classification and paraphrasing – using a common evaluation framework and standardized texts.

6.3.1 Few-shot prompting

The few-shot provides a model with a small number of task-specific examples, each illustrating a clear input-output pair. This approach relies on the model’s ability to infer task patterns from given examples and apply them to new inputs. The number of examples was fixed at two per task to balance instructional clarity and token efficiency.

An example of a few-shot classification prompt is shown below:

Classify the sentiment:

Example 1: Text: I love this → Positive

Example 2: Text: This was terrible → Negative

Text: The customer service was okay →

The corresponding implementation is in the `prompt_builder.py` module under the `render_few_shot()` function (Figure 10). The summary and paraphrasing tasks used similar structures with task-specific phrasing and examples drawn from synthetic reference data.

```
def render_few_shot(input_text, task_type):  
    if task_type == "classification":  
        return (  
            "Classify the sentiment:\n"  
            "Example 1: Text: I love this → Positive\n"  
            "Example 2: Text: This was terrible → Negative\n"  
            f"Text: {input_text} →"  
        )
```

Figure 10. Few-shot prompting

6.3.2 Chain-of-Thought prompting

The chain-of-Thought prompting guides the model through step-by-step reasoning process by presenting intermediate steps before reaching a final answer. This technique is particularly useful for task that benefit from decomposition method or an explicit reasoning path (Wei et al., 2023).

An example of CoT prompt for paraphrasing is shown below:

Original: I need to find a better way to manage my time.

Step 1: Identify the meaning

Step 2: Rephrase using different words.

Paraphrase:

The implementation of this logic is embedded in the `render_chain_of_thought()` function (Figure 11).

```
def render_chain_of_thought(input_text, task_type):
    if task_type == "summarization":
        return (
            "Let's summarize step by step:\n\n"
            f"Text: {input_text}\n\n"
            "Step 1: Identify key ideas.\nStep 2: Combine into a single sentence.\nSummary:"
        )
    elif task_type == "classification":
        return (
            f"Text: {input_text}\n"
            "Let's analyze the sentiment step by step.\n"
            "Is the tone positive, neutral, or negative?\nAnswer:"
        )
    elif task_type == "paraphrasing":
        return (
            f"Original: {input_text}\n"
            "Step 1: Identify the meaning.\n"
            "Step 2: Rephrase using different words.\nParaphrase:"
        )
```

Figure 11. Chain-of-Thought prompting

6.3.3 Evaluation and comparison

Both ICL strategies were integrated into the same evaluation pipeline used with structured and zero-shot prompts. Each test use case was evaluated using BLEU, ROUGE-L and METEOR metrics and logged for later aggregation. As will be shown later in Chapter 6, the few-shot prompting consistently outperformed CoT on paraphrasing tasks, while CoT showed more consistent results on classification tasks when used with GPT-4o.

The use of ICL strategies was crucial to understanding how LLMs manage contextual information embedded within the prompt itself and allowed the thesis to simulate more realistic prompting scenarios, in which examples or chains of reasoning are natural part of the task instructions.

6.4 Environment setup and authentication

OpenAI's API key was securely managed using a .env file and the dotenv package, allowing sensitive credentials to be separated from the codebase (Figure 12). The setup below initializes the client object with appropriate authentication, allowing queries to be made to selected models as specified in the script.

```
from openai import OpenAI
from dotenv import load_dotenv
import os

# Load API key
load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
```

Figure 12. Client object authentication

6.4.1 Experiment control and model switching

The selected model was specified using a global configuration variable (model_name), which allows the user to switch between GPT-3.5-turbo and GPT-4o for comparison (Figure 13).

```
model_name = "gpt-4o" #or "gpt-3.5-turbo"
```

Figure 13. Model switching

The following API call was made using OpenAI's SDK version 1.x, which replaced legacy methods (Figure 14): The figure shows Python code that uses OpenAI's chat.completions.create method for generative AI chat. Prompt generation was managed by the render_prompt() dispatcher function, which routed each request to the appropriate variant based on the task and prompt strategy.

```
response = client.chat.completions.create(  
    model=model_name,  
    messages=[{"role": "user", "content": prompt}],  
    temperature=0.7,  
    max_tokens=200  
)
```

Figure 14. API call

The user input is entered as a message list, and the following parameters are set for the model:

- Model=model_name: specifies the model to use (e.g. "gpt-3.5-turbo"),
- Messages= [{"role": "user", "content": prompt}]: the message history of the chat, here a single user message,
- Temperature=0.7: adjusts the degree of creativity (0= deterministic, 1= more random),
- Max_tokens=200: limits the maximum length of the response to 200 tokens.

6.4.2 Execution workflow

The workflow was designed to iterate through each test input from a JSON dataset, apply each defined prompting strategy (e.g., zero-shot, few-shot, CoT, structured) and perform the evaluations accordingly. The result of each combination was saved in a structured JSONL log (Figure 15).

```
# Run experiment
for entry in test_data:
    input_text = entry["input"]
    reference = entry["reference"]
    task = entry["task"]
    complexity = entry.get("complexity", "unknown")
    domain = entry.get("domain", "unknown")

    for strategy in ["zero-shot", "few-shot", "cot", "structured"]:
        try:
            print(f"🚀 Running {task} with {strategy}...")
            prompt = render_prompt(input_text, strategy, task)
            output, tokens_used = query_openai(prompt)
            scores = evaluate_output(output, reference)

            result = {
                "timestamp": datetime.now().isoformat(),
                "task": task,
                "prompt_type": strategy,
                "complexity": complexity,
                "model": model_name,
                "prompt": prompt,
                "output": output,
                "reference": reference,
                "BLEU": scores["BLEU"],
                "ROUGE": scores["ROUGE"],
                "METEOR": scores["METEOR"],
                "tokens_used": tokens_used
            }

            with open("outputs/result_log.jsonl", "a", encoding="utf-8") as log_file:
                log_file.write(json.dumps(result, ensure_ascii=False) + "\n")

            print(f"✅ Completed {task} with {strategy}")

        except Exception as e:
            print(f"❌ Error during {task} with {strategy}: {e}")
```

Figure 15. Experiment loop

All results were appended to a JSON Lines file, `result_log.jsonl`, in the `outputs` directory. Each log entry included a timestamp, task, strategy, model name, prompt, generated output, evaluation scores and tokens used. The framework included error handling to catch API or encoding-related exceptions. Failed executions were logged to the console for debugging purposes and did not halt the entire experiment. Encoding issues (e.g., those caused by special characters such as “→”) were resolved by forcing UTF-8 encoding in file write operations.

At least five examples per task were used to evaluate the four prompting strategies for both models. This resulted in a total of over 750 test runs, allowing for metric-based comparisons of model behaviour and prompt effectiveness. The test suite was repeatable and backed-up logs ensured traceability of the results for later evaluation in Chapter 6.

6.4.3 Implementation of automated metric-based evaluation

To systematically evaluate the quality of the outputs of different prompt strategies and tasks, an automated evaluation module was implemented using three widely known natural language generation (NLG) metrics: BLEU, ROUGE-L and METEOR. These metrics were chosen based on their complementary properties in evaluating syntactic overlap, n-gram similarity and semantic adequacy.

All three metrics were calculated using the evaluation library provided by Hugging Face, ensuring reproducibility and consistency across test runs. The implementation was integrated as follows (Figure 16) and was called immediately after the model output was generated and used to build a structured result log.

```
from evaluate import load

# Load evaluation metrics
bleu = load("bleu")
rouge = load("rouge")
meteor = load("meteor")

# Evaluate output
def evaluate_output(prediction, reference):
    return {
        "BLEU": bleu.compute(predictions=[prediction], references=[reference])["bleu"],
        "ROUGE": rouge.compute(predictions=[prediction], references=[reference])["rougeL"],
        "METEOR": meteor.compute(predictions=[prediction], references=[reference])["meteor"]
    }
```

Figure 16. Hugging Face evaluate

6.4.4 Logging and score tracking

Each result was saved in a JSON Lines (.jsonl) file and included the complete prompt, model output, reference, prompt type, task and computed metric scores. This allowed the results to be later grouped and analyzed by model, strategy and task as discussed later (Figure 17). The result log not

only enabled repeatability but also formed the basis for visual observations and metric-based comparisons between different prompt engineering strategies.

```
{
  "timestamp": "2025-04-22T19:04:46.407697", "task": "paraphrasing", "prompt_type": "zero-shot", "model": "gpt-4o", "prompt":
  "Paraphrase the following:\n\nCan you help me understand how this algorithm works?", "output":
  "Could you assist me in comprehending the functioning of this algorithm?", "reference": "Could you explain the functioning of this algorithm to me?",
  "BLEU": 0.3882726777522233, "ROUGE": 0.6666666666666666, "METEOR": 0.775219664108553
}
```

Figure 17. Example jsonl structure

6.5 Evaluating the framework on real-world natural language processing tasks

To validate the effectiveness and flexibility of the developed prompt engineering framework, a comprehensive set of experiments was conducted on three representative natural language processing tasks: summarization, sentiment classification and paraphrasing. These tasks were chosen to cover a wide range of cognitive demands – from extracting key content (summarization) to making discrete decisions (classification) and generating semantically equivalent variations (paraphrasing). The structured development and evaluation of the framework was supported by three curated datasets, each corresponding to a specific NLP task. The detailed structure of these datasets, including example inputs, expected outputs, and task-specific instructions, is presented in Appendices 1-3.

Each task was evaluated with four different prompt strategies – zero-shot, few-shot, CoT and structured – using inputs designed to reflect common real-world use cases. The evaluation was conducted using both GPT-3.5-turbo and GPT-4o, allowing for performance analysis across different models.

6.5.1 Use case 1: Summarization

In the summarization task, the model was asked to generate concise summaries of short to medium length informative texts. The input examples were designed to simulate prescriptive, descriptive and sentences, each paired with a reference summary (Table 5). This task assessed the model’s ability to abstract meaning and concisely reformulate content. The outputs were evaluated for accuracy, brevity and semantic alignment.

Table 5. Summary use case example

Example input	Reference summary
<i>Artificial intelligence is being increasingly adopted across industries for tasks like automation, data analysis, and personalization.</i>	<i>AI is widely used for automation, analytics, and personalization.</i>

6.5.2 Use case 2: Sentiment classification

In the classification task, user-generated texts were labelled with one of three sentiments: *positive, neutral or negative* (Table 6). The input set consisted of short review-style sentences designed to assess the model’s contextual understanding and ability to make discrete label decisions. This task highlighted how different prompting strategies affect the reliability and consistency of classification decisions.

Table 6. Classification use case example

Example input	Reference label
<i>The customer service was okay. Nothing special, but nothing terrible either.</i>	<i>Neutral</i>

6.5.3 Use case 3: Paraphrasing

In the paraphrasing task, the model had to reformulate a given sentence using alternative wording while maintaining the original meaning (Table 7). The inputs were generated to be relevant to everyday communication, from professional to academic. The evaluation focused on semantic accuracy, fluency and variation from the source text.

Table 7. Paraphrasing use case example

Example input	Reference paraphrase
<i>Can you help me understand how this algorithm works?</i>	<i>Could you explain the functioning of this algorithm to me?</i>

6.5.4 Dataset format and test distribution

All test inputs and reference outputs were defined in a JSON files (Appendix 1: test_inputs, Appendix 2: test_inputs_extended and Appendix 3: test_inputs_complexity_rotated) structured as follows (Figure 18). Each task was completed with three to nine examples of varying complexity, resulting in a total of 69 test entries. When running test runs with four prompt types and two models, the total number of test runs exceeded 750, providing sufficient data for aggregated metrics analysis. The exact test distribution and dataset formatting used in each task are presented in Appendices 1-3, including input-output samples and reference completions.

```
{
  "task": "paraphrasing",
  "input": "The seminar was informative and covered the topic in depth.",
  "reference": "The workshop provided detailed and insightful information on the topic."
}
```

Figure 18. Test input

7 Results and analysis

7.1 Task design and example use cases

To evaluate the effectiveness of different prompt engineering strategies, this study implemented a controlled testing framework across three core natural language processing tasks: text summarization, sentiment classification and paraphrasing. Each task was selected based on its relevance to real-world applications and its ability to challenge different aspects of language model reasoning, semantic compression and interpretability.

All use cases were evaluated with four different prompting strategies and executed using OpenAI's GPT-3.5-turbo and GPT-4o models via the OpenAI API. Prompts were standardized by strategy and adjusted only in terms of their intended technique and task relevance, allowing for a comparative evaluation of prompt behaviour under otherwise identical conditions. Each strategy was applied to the same input examples to ensure consistency across outputs. This design allowed for a detailed analysis of how different prompting styles affect the model's language generation behaviour and how interaction between strategy, task and model affects performance.

As shown in (Table 8), tasks were implemented at varying linguistic complexities to assess the scaling of strategies across difficulties. A use case analysis of the results is provided in Sections 6.2.1 – 6.2.3.

Table 8. Summarization prompting techniques and examples

Prompt strategy	Task definition	Input example (Summarization)
Zero-shot	Directly instruct the model without examples.	<i>Summarize the following:</i> <i><input_text></i>
Few-shot	Provide a few annotated examples before the actual task.	<i>Example 1:</i> <i>Text: AI helps automate tasks.</i> <i>Summary: AI is used for automation.</i> <i>...</i> <i>Text: <input_text></i> <i>Summary:</i>
CoT	Encourage reasoning steps before giving the last answer.	<i>Let's summarize step by step</i> <i>Text: <input_text></i> <i>Step 1: Identify key ideas.</i> <i>Step 2: Combine them.</i> <i>Summary:</i>
Structured	Explicitly structure the prompt using task metadata.	<i>Task: Summarization</i> <i>Input: <input_text></i> <i>Instructions: Summarize in one sentence.</i> <i>Output:</i>

7.2 Metric-based evaluation of prompt strategies

This section presents selected use cases and a comparative evaluation of strategies for three different NLP task types. Each use case was evaluated using four prompt variants applied to both GPT-3.5-turbo and GPT-4o models. The results were scored using the BLEU, ROUGE-L and METEOR metrics. The

The results were generated in a fully controlled evaluation environment, where all tests were run with consistent input and model parameters (Appendices 1-3). The evaluation process was fully automated and total of 761 samples were generated and stored in a structured format for further analysis. The number of tests was determined empirically; after several evaluation batches, it was observed that there were no more significant changes in the results, indicating that the stability of the results was sufficient for the accuracy of the analysis. The model generation parameters used were a temperature of 0.7 and a maximum response length of 200 tokens, which were chosen as a compromise between creativity and reproducibility of the outputs. In the visual analysis METEOR scores were weighted because they proved to be particularly sensitive to semantic similarity and highlighted differences between task strategies, especially for short-word outputs (Figure 19).

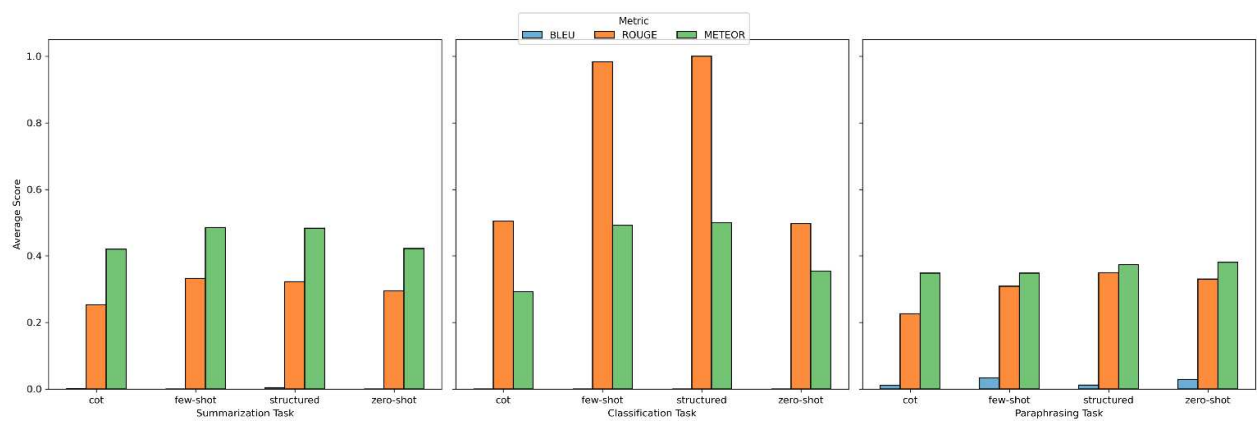


Figure 19. Evaluation metric average scores per task

7.2.1 Use case 1: Text summarization

The evaluation of prompting strategies used in text summarization was performed using a comprehensive set of input samples extracted from predefined test datasets. The summary tasks aimed to measure the quality, consistency and token efficiency of the outputs produced by different prompting techniques: zero-shot, few-shot, CoT and structured prompting. Each test use case was presented according to the models using prompts constructed to the logic defined in the `prompt_builder.py` module.

Zero-shot prompts instructed the models directly to summarize the input, while few-shot prompts presented examples before requesting the summary. CoT prompts divided the summary task into intermediate steps and structured prompts used a fixed input-output command format. A total of 255 summary prompts executions were logged. Approximately 69.4% of the summary tasks were related to complexity and subject area metadata, allowing for partial segmentation of the results according to these factors. The key statistical results are summarized in (Table 9), which presents the average evaluation scores of the summary task across the four strategies evaluated.

Table 9. Average evaluation metrics for summarization by prompt type

Prompt type	BLEU	ROUGE-L	METEOR
cot	0.135	0.419	0.404
few-shot	0.166	0.461	0.455
structured	0.188	0.486	0.471
zero-shot	0.103	0.365	0.354

Preliminary analysis revealed clear trends across strategies. Structured prompting generally achieved higher ROUGE-L and METEOR scores, indicating strong alignment with the reference

summaries. Few-shot prompting consistently performed better than zero-shot, suggesting that providing examples improves model comprehension even for relatively short summaries. Chain-of-thought prompting yielded mixed results; while useful for complex summaries, it sometimes introduced unnecessary verbosity, which affected accuracy-based scores such as BLEU score.

The token efficiency analysis revealed notable differences between the strategies (Figure 20). Zero-shot prompts were the most economical in token usage, while CoT prompts, by design, consumed much more tokens due to their stepwise reasoning structure. Structured prompts balanced token efficiency and quality.

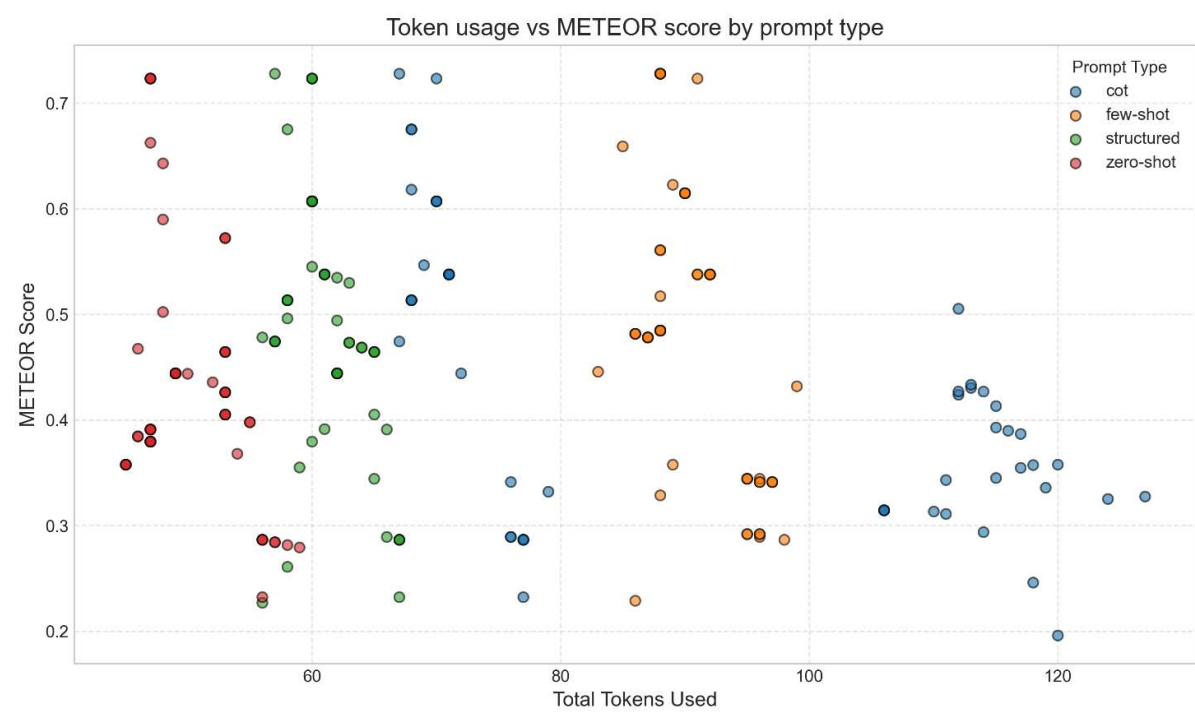


Figure 20. Token efficiency vs. summarization quality

Structured prompting had the best median performance, closely followed by few-shot prompting. These findings suggest that for text summarization tasks, structured and few-shot prompting provide better quality, with structured prompting providing a particularly effective balance between performance and cost. However, task-specific conditions, such as summary complexity and desired brevity, must be considered when choosing the optimal prompting approach.

7.2.2 Use case 2: Sentiment classification

The sentiment classification tasks were assessed in the same way as summary tests, using the four established prompting strategies. The test inputs were derived from predefined datasets covering multiple complexity levels and domains. Each sentiment classification input was transformed into prompts according to templates defined in the `prompt_builder.py` module. Zero-shot prompts directly requested a sentiment labelling, few-shot prompts provided annotated examples before presenting the new input, chain-of-thought prompts guided the model through a reasoning process and structured prompts followed a task-instruction-output format.

As in the summary tasks, two different model versions were used in the sentiment classification experiments: GPT-3.5-turbo and GPT-4o. The model outputs were evaluated using BLEU, ROUGE-L and METEOR scores, despite the known limitations of such metrics in pure classification tasks. In addition, qualitative error analysis was performed to complement findings based on the metrics.

A total of 278 sentiment classification prompt runs were logged, representing approximately 36.5% of all results collected. GPT-4o was observed to consistently produce higher METEOR scores across all prompt types compared to GPT-3.5-turbo, indicating improved semantic alignment and classification robustness.

Structured and few-shot prompting showed the most consistent and best-performing results (Figure 21), with METEOR scores tightly around 0.5, indicating strong semantic alignment with reference labels. The zero-shot prompting showed slightly lower median performance and a wider distribution of scores, reflecting random variations in the quality of outputs. In contrast, CoT prompts showed considerable variability, with scores spanning the entire range from near zero to 0.5. This inconsistency suggests that CoT prompts may introduce unnecessary complexity into the classification contexts, which may reduce clarity rather than improve inference. The results confirm the suitability of the structured and few-shot prompts for tasks requiring categorical decisions, such as sentiment classification.

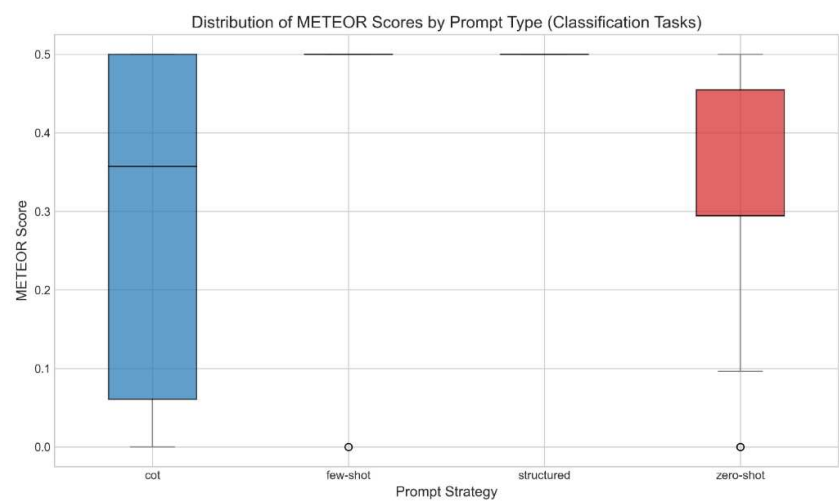


Figure 21. METEOR by prompt type (classification)

Token efficiency analysis for classification tasks showed more limited token usage compared to summarization, especially in structured and zero-shot strategies. As expected, CoT prompts consumed the most tokens due to the staggered inference structure. As shown in (Figure 22), GPT-4o maintains higher classification quality with varying token budgets, while GPT-3.5-turbo displayed a steeper decline in quality as token usage decreased.

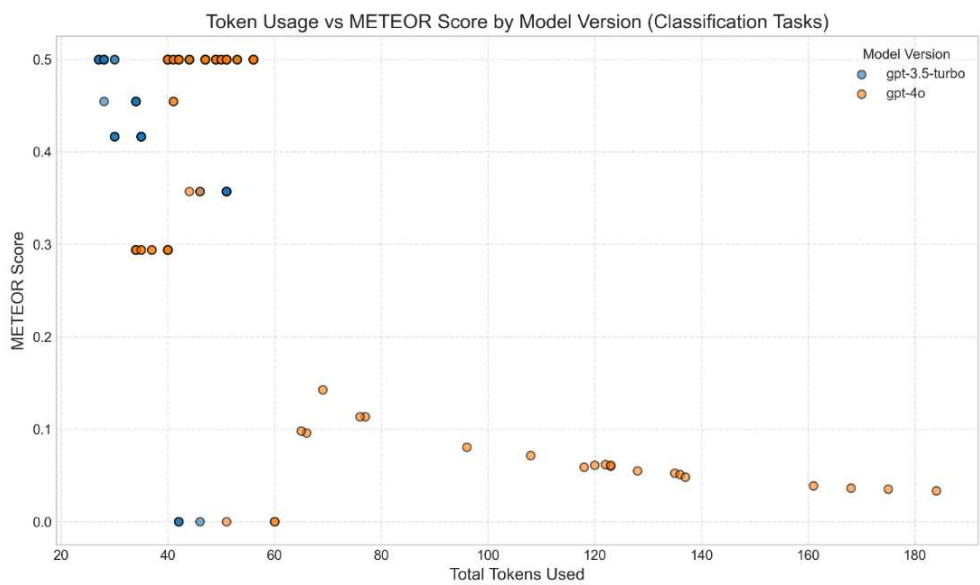


Figure 22. Token usage against METEOR score (classification)

The results highlight that structured and few-shot prompts are most effective in sentiment classification tasks, especially when used with the new GPT model. Furthermore, the findings show that the choice of model version significantly affects classification reliability even under identical prompt engineering conditions.

7.2.3 Use case 3: Text paraphrasing

The final use case evaluated in the test phase was text paraphrasing, which required models to generate semantically equivalent reformulations of given input sentences. As in the previous tasks, four prompt strategies were used. Paraphrasing input samples were drawn from all three test sets and included varying levels of complexity and subject area context. Paraphrasing tasks were evaluated using the same triad of metrics: BLEU to capture n-gram overlap, METEOR to assess synonym substitution and word order flexibility and ROUGE-L to measure recall and phrase-level alignment.

Across all prompt types, structured prompting showed the most consistent performance, producing paraphrases that retained original meaning while closely following the instructional format. This consistency was reflected in higher ROUGE-L and BLEU scores, indicating strong coverage and

semantic retention. Few-shot prompting also performed well, particularly in capturing idiomatic phrasing, but showed slightly more variation in structural accuracy.

The distribution of paraphrasing quality across prompt strategies is visualized in (Figure 23), where the structured and few-shot prompt shows tight and elevated METEOR score distribution compared to zero-shot and CoT strategies. It is noteworthy that the CoT prompts produced mixed results in paraphrasing tasks. While the step-by-step reasoning format was sometimes useful for longer and more abstract inputs, it often led to unnecessary elaboration or restatement rather than true paraphrasing. This tendency negatively affected BLEU and METEOR scores due to semantic drift or overextension.

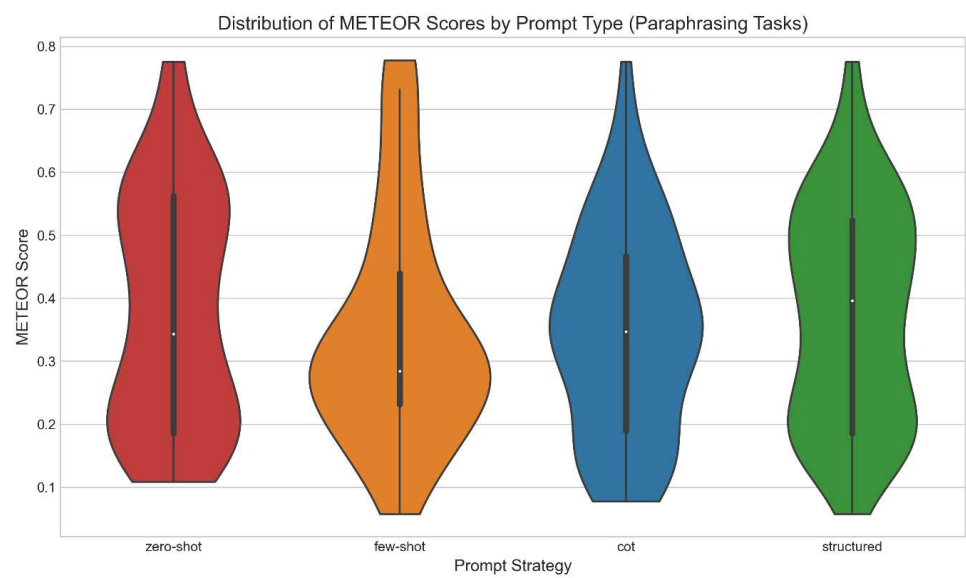


Figure 23. METEOR by prompt type (paraphrasing)

Zero-shot prompting produced acceptable outputs, but it generally performed worse than other strategies, especially in preserving precise meaning or lexical variation. It often produced paraphrases that were too literal or not sufficiently differentiated from the input.

The token efficiency analysis of the paraphrasing tasks, shown in (Figure 24), highlighted that structured prompts offered a favourable balance between the number of instructions and the

quality of output. As expected, CoT prompts were the most token-intensive but did not produce proportional improvements in semantic accuracy.

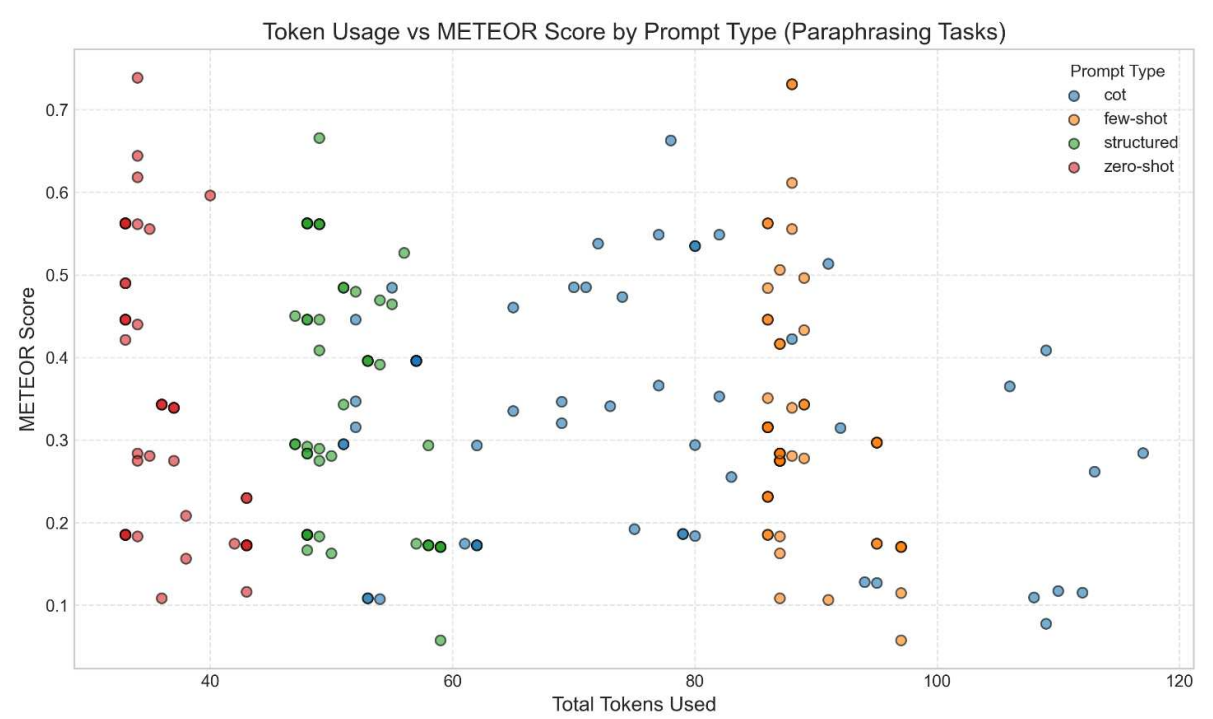


Figure 24. Token usage vs. METEOR score (paraphrasing)

The following (Figure 25) shows a heatmap showing the average scores for different evaluation metrics (BLEU, ROUGE-L, METEOR) for different prompt strategies in the paraphrasing tasks. The analysis of the figure is carried out below.

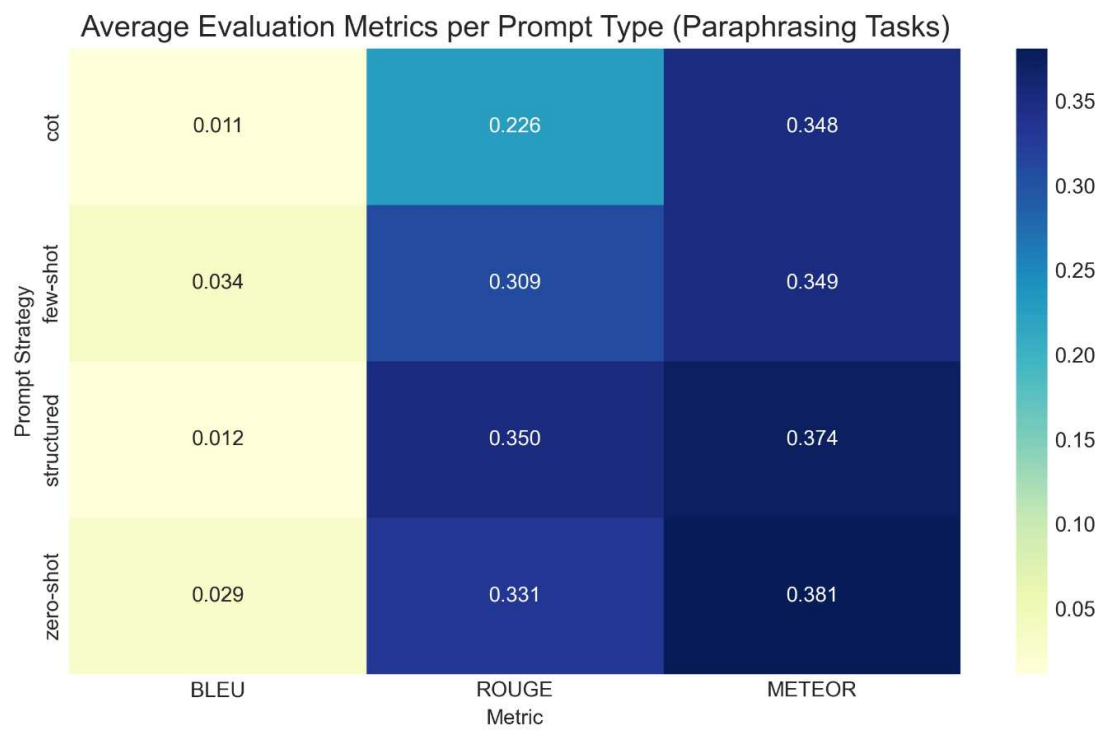


Figure 25. Average BLEU, ROUGE, METEOR per prompt type (paraphrasing)

The zero-shot strategy performed best overall on the paraphrasing tasks, especially on the METEOR metric (0.381) and ROUGE value (0.331), indicating that the model can perform effectively without example inputs when the instructions are clear. The structured prompt produced the best ROUGE score (0.350), suggesting that clearly structured instructions help the model preserve the structure and content of the original text. In contrast, the chain-of-thought strategy clearly lagged on all evaluation metrics, suggesting that multistep reasoning is not well suited to paraphrasing, which requires more concise and direct formulation. The results show that while providing examples (few-shot) can be beneficial, simple and structured approaches can be equally or even more effective for certain NLP tasks.

7.3 Statistical evaluation and key findings

Following the individual task evaluations, a statistical synthesis across tasks was conducted to better understand the overall performance patterns of the different prompting strategies (Figure 26). This broader analysis focuses on performance consistency, efficiency and metric alignment across previously analysed use cases.

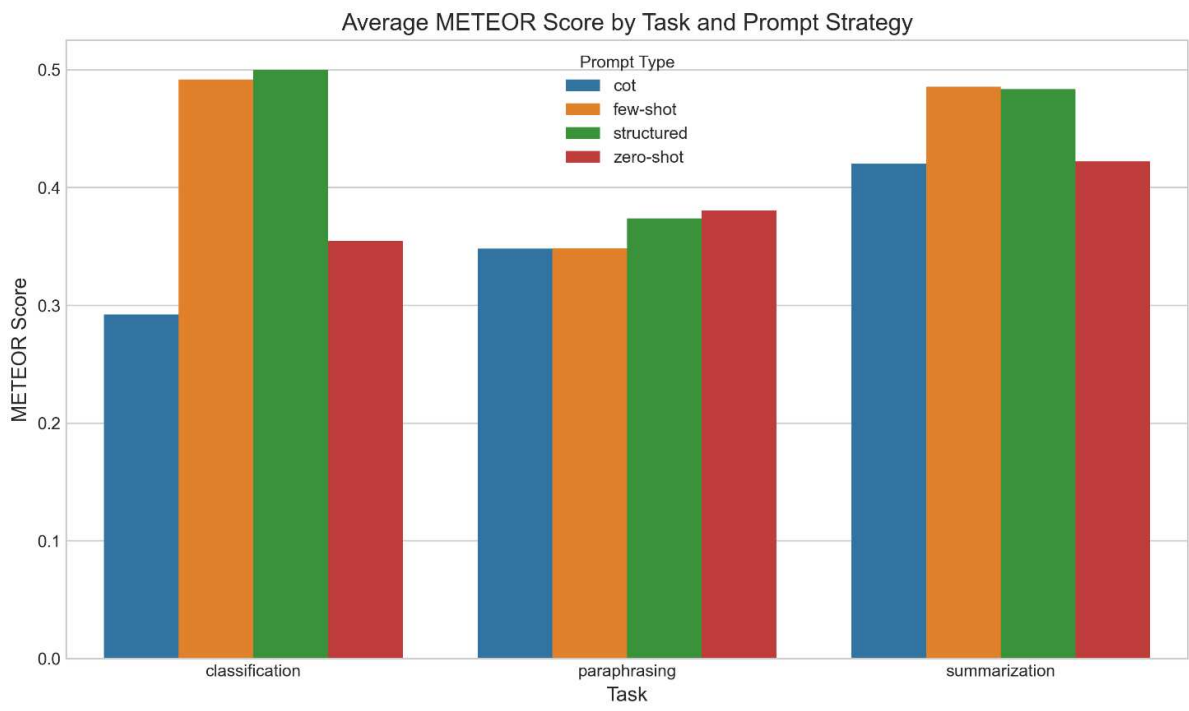


Figure 26. Cross-task comparison of strategies

The bar chart shows that among the prompt strategies, structured and few-shot produce the best results on the METEOR scale across tasks. In classification tasks, these strategies achieve the highest score (around 0.50), while zero-shot and especially CoT lag far behind. In paraphrasing tasks, the differences between the different strategies are smaller, but structured and zero-shot produce slightly better results than the others. In summarization, structured and few-shot again perform best, while CoT is slightly weaker. The results suggest that the model’s performance improves when the prompt is either example-driven or structured, while chained reasoning is not an effective strategy for tasks that require concise and precise linguistic production.

In addition to raw performance, token efficiency was also evaluated to understand the cost-effectiveness of the different strategies. As shown in (Figure 27), zero-shot prompts achieved the highest METEOR value per token due to minimal token usage. Structured prompting offered the best balance between semantic quality and efficiency, outperforming more complex strategies like few-shot and CoT prompts. Few-shot prompts showed moderate efficiency, while CoT prompts, despite its intended reasoning benefits, showed the lowest efficiency due to its verbose generation style.

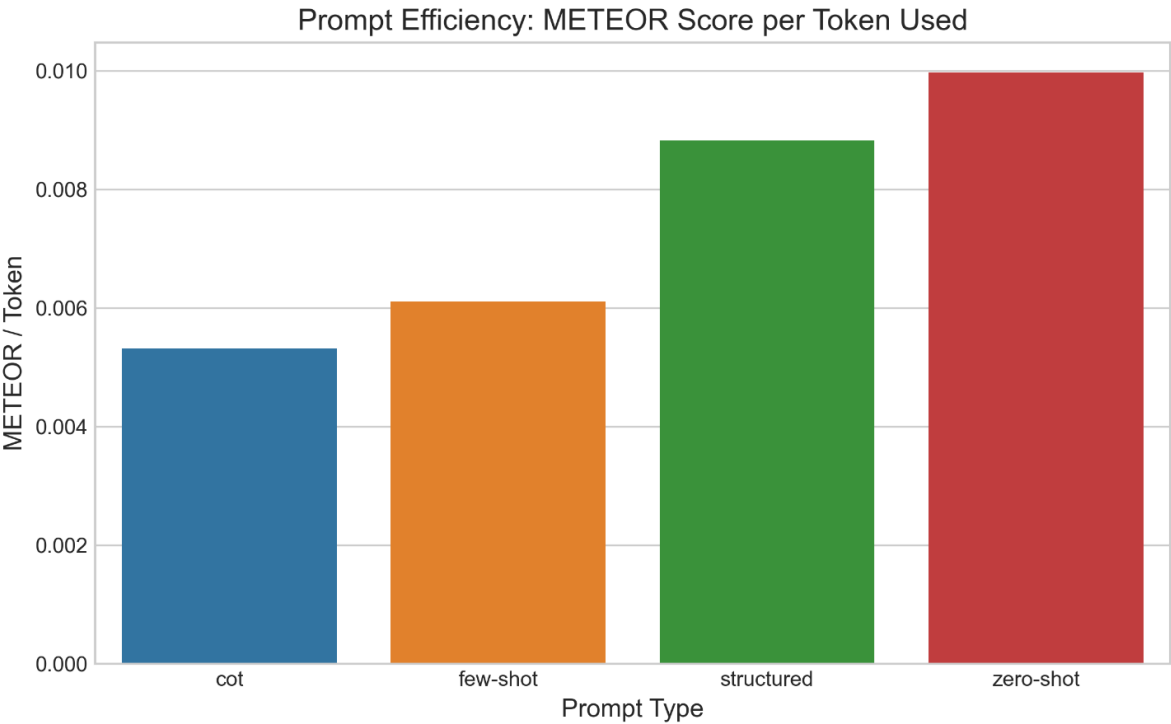


Figure 27. METEOR per token

To validate the choice of metrics and understand the relationships between the metrics, a correlation analysis was performed between BLEU, ROUGE and METEOR scores across all tasks. The correlation matrix shown in the (Figure 28) reveals significant differences between the evaluation metrics. The BLEU metric has a weak correlation with ROUGE (0.07) and METEOR (0.25), suggesting that BLEU assesses language quality differently than the other metrics – in particular, emphasizing word order and accurate n-gram matches. In contrast, the ROUGE and METEOR metrics show a reasonably strong correlation (0.60), suggesting that they emphasize some of the same features, such as content coverage and linguistic naturalness. This highlights that the choice of evaluation metric directly affects the performance of the model, and that the BLEU metric is particularly useful for tasks where word order is important.

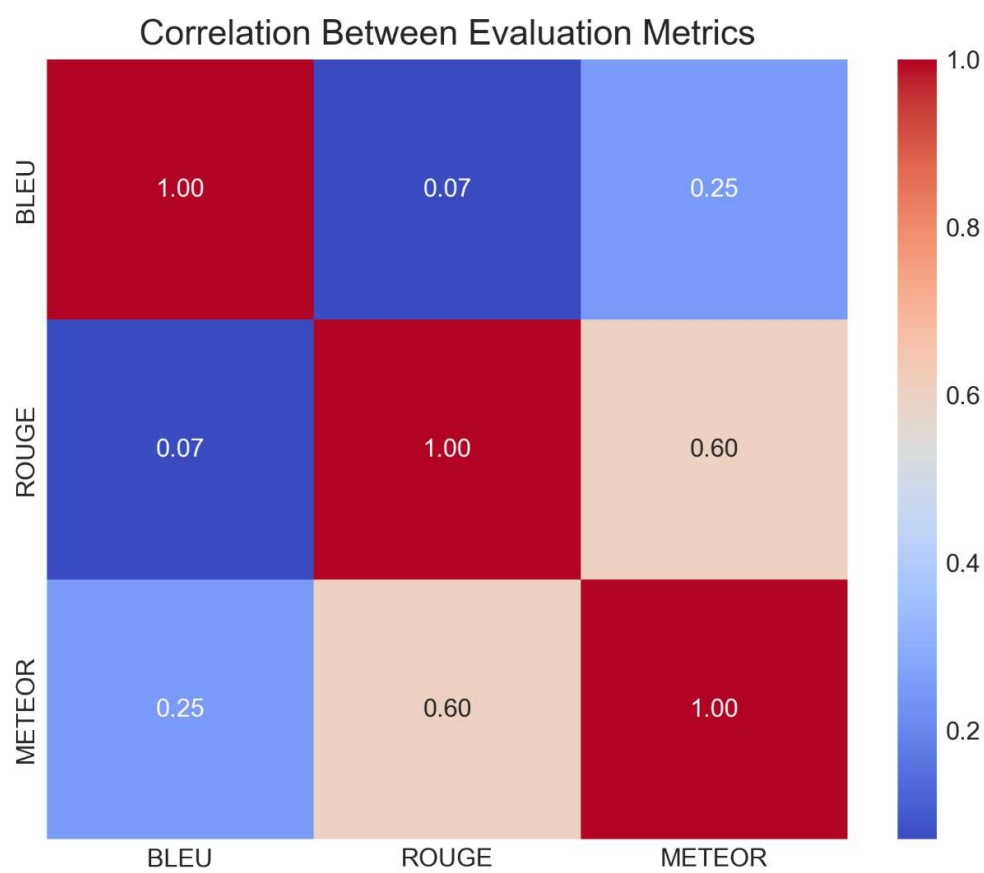


Figure 28. Correlation between BLEU, ROUGE and METEOR

8 Conclusions

8.1 Summary and design science contributions

The results of this research show that different prompting strategies can affect the quality and efficiency of the content produced by large language models. In a comparison of three practical tasks: summarization, classification and paraphrasing, structured prompting performed best on average, especially in terms of quality and predictability of results. Few-shot prompting brought creativity, but slightly less consistency. Zero-shot was effective in the use of tokens but was less precise in terms of content. The chain-of-thought strategy proved to be unstable and prone to over-explanation, especially in paraphrasing tasks.

In direct response to the main research question, the comparison showed that no strategy is unambiguously best in all situations, but structured prompting offered the strongest balance across metrics and tasks. This supports the hypothesis that clearly structured prompting improves the model's understanding of the task and better targets the output. Statistical evaluation confirmed that the structured approach produces the best quality-to-token ratio and the most consistent results across metrics.

The strengths of structured prompting were based on its logical structure, which combined the task, the example and the desired response format into a clear entity. This helped the models understand the user's intent and reduced the scope for interpretation, which improved the quality consistency of the results. Although automatic measures do not always capture nuances of language or alternatively acceptable responses, especially in creative tasks, structured prompting proved to be the most effective and scalable strategy in this research, working without task specific adjustments.

From a practical perspective, the research provided clear guidance on how to utilize LLM: structured prompting is best suited for situations where accuracy, repeatability and semantic consistency are needed such as in abstracts, policy drafts or when designing customer service automation. In relation to sub-question 1, which focused on the influence of in-context learning strategies on prompt optimization outcomes, the results suggest that few-shot is useful when the

output requires a personal touch or stylistic variation. Zero-shot can be viable solution in situations where speed and cost-effectiveness are more important than perfect content accuracy.

In response to sub-question 2 regarding the evaluation of prompt effectiveness, the research was based on the Design Science Research approach and produced an evaluation framework that includes a systematic way to create and manage different prompts, an experimental environment using OpenAI models, qualitative evaluation metrics (BLEU, ROUGE-L, METEOR), among which METEOR was identified as the most sensitive and reliable indicator of prompt strategy effectiveness. In addition, visual analysis tools were developed to support interpretation. This whole creates a reusable and extensible model that is suitable for both research and development.

8.2 Research limitations and areas for development

Although this study provides clear evidence of the impact of different prompting strategies on the performance of LLM models, it had several limitations that affect several limitations that affect the generalizability of the results. First, the study was limited to two models – GPT-3.5-turbo and GPT-4o – and the results cannot be directly applied to other architectures or smaller models without further validation, as differences in for example, training data or tokenization may affect how the models respond to different prompt structures.

Second, the evaluation was based solely on automated metrics (BLEU, ROUGE-L, METEOR), which are not always sufficient for assessing creative or more open-ended tasks such as paraphrasing. Human-based evaluation would add value, especially in assessing language fluency, tone and contextual appropriateness. Third, the data was limited to English tasks, which limits conclusions about multilingual environments where language structure and semantic sensitivity may vary. Furthermore, the tasks represented general use cases, but applications in specialized fields such as law, medicine or economics were not considered.

Fourth, the research did not address adaptive or dynamic prompt optimization methods such as RLHF, RAG or prompt fine-tuning based on external feedback, which could improve the effectiveness of structured prompting. Fifth, all strategies were tested in a static format and iterative or multi-stage fine-tuning was not utilized, even though it could have a significant impact on, for example, the quality of summarization or paraphrasing. These limitations do not weaken the key

findings of the research but clearly indicate where further research is needed to better understand the functionality and flexibility of prompt strategies in increasingly complex LLM application.

8.3 Future research directions

Based on the results of this research, different prompting strategies can be recommended for different task environments. The results reinforce the notion that a well-designed prompt improves the efficiency and predictability of LLM models. Prompt design is no longer limited to parameter adjustments or model training but is essentially related to the design of human-AI interaction. The goal was to show that structured prompt design can improve the quality, consistency and cost-effectiveness of outputs.

In the future, attention should be paid to how prompts could guide the model to express uncertainty in situations where information is incomplete. Promising approaches include, for example, breaking questions into stages and guiding the model to assess the reliability of its answers. This could be supported by AI-assisted systems that identify unreliable answers and offer alternative structures.

References

- Arize AI. (2023). *Prompt optimization & few-shot prompting: From manual tuning to programmatic pipelines*. Arize AI Blog. <https://arize.com/blog/prompt-optimization-few-shot-prompting/>
- Bach, S., He, W., Rothmel, J., & Shafran, I. (2022). *PromptSource: Creating, sharing, and using natural language prompts*. arXiv preprint [arXiv:2202.01279](https://arxiv.org/abs/2202.01279)
- Banerjee, S., & Lavie, A. (2005). *METEOR: An automatic metric for MT evaluation with improved correlation with human judgments*. In *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization* (pp. 65-72). <https://aclanthology.org/W05-0909/>
- Boucharenc, I., Gerald, T., Ghannay, S., Servan, C., & Rosset, S. (2024). *Impact of prompt on latent representations in LLMs*. Manuscript submitted for publication. <https://open-review.net/pdf?id=10kBEqYKKN>
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). *Language models are few-shot learners*. arXiv preprint <https://arxiv.org/abs/2005.14165>
- Deng, M. (2023, February 24) RLPrompt: Optimizing Discrete Text Prompts with Reinforcement Learning. LTI/CMU Blog <https://blog.ml.cmu.edu/2023/02/24/rlprompt-optimizing-discrete-text-prompts-with-reinforcement-learning/#:~:text=TL%3BDR%3A%20Prompting%20enables%20large%20language,tasks%20using%20our%20library%20here>
- Deng, S., Huang, Y., Li, C., Dong, Z., Chen, X., Zhang, J., ... & Sun, M. (2022). *RLPrompt: Optimizing discrete text prompts with reinforcement learning*. arXiv preprint [arXiv:2205.12548](https://arxiv.org/abs/2205.12548)
- Evidently AI. (2025, February 25(Samuylova?). *LLM evaluation metrics and methods, explained simply*. Evidently AI. <https://www.evidentlyai.com/llm-guide/llm-evaluation-metrics>
- Farquhar, S., Kossen, J., Kuhn, L., & Gal, Y. (2024). Detecting hallucinations in large language models using semantic entropy. *Nature*, 630, 625–630. <https://doi.org/10.1038/s41586-024-07421-0>
- Furniturewala, S., Zhang, W., Rajani, N. F., & Gardner, M. (2024). *Thinking fair and slow: On the efficacy of structured prompts for debiasing language models*. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 204–221). Association for Computational Linguistics. <https://aclanthology.org/2024.emnlp-main.13/>
- Guo, X., & Vosoughi, S. (2024). Serial position effects of large language models. arXiv preprint. <https://arxiv.org/abs/2406.15981>
- Hasan, Z. (2023, November 19). *Lost in the middle: How language models use long contexts*. Weaviate. <https://weaviate.io/papers/paper-2>
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105. <https://doi.org/10.2307/25148625>

- Hevner, A., vom Brocke, J., & Maedche, A. (2020). Introduction to design science research. In J. vom Brocke, A. Hevner, & A. Maedche (Eds.), *Design science research. Cases* (pp. 1–13). Springer. https://doi.org/10.1007/978-3-030-46781-4_1
- Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., ... & Fung, P. (2023). *Survey of hallucination in natural language generation*. ACM Computing Surveys, 55(12), 1–38. [arXiv:2202.03629](https://arxiv.org/abs/2202.03629)
- Jiang, Z., Xu, F. F., Araki, J., & Neubig, G. (2020). *How can we know what language models know?* Transactions of the Association for Computational Linguistics, 8, 423–438. [arXiv:1911.12543](https://arxiv.org/abs/1911.12543)
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., & Amodei, D. (2020). *Scaling laws for neural language models*. arXiv preprint. <https://arxiv.org/abs/2001.08361>
- Khatun, A., & Brown, D. G. (2023). Reliability check: An analysis of GPT-3's response to sensitive topics and prompt wording. In A. Ovalle, K.-W. Chang, N. Mehrabi, Y. Pruksachatkun, A. Galystan, J. Dhamala, A. Verma, T. Cao, A. Kumar, & R. Gupta (Eds.), *Proceedings of the 3rd Workshop on Trustworthy Natural Language Processing (TrustNLP 2023)* (pp. 73–95). Association for Computational Linguistics. <https://aclanthology.org/2023.trustnlp-1.8/>
- Kojima, T., Gu, S. S., Reid, M., Yao, Y., & Schölkopf, B. (2022). *Large language models are zero-shot reasoners*. arXiv preprint [arXiv:2205.11916](https://arxiv.org/abs/2205.11916)
- Kosinski, M., & Forrest, A. (2024, March 26). What is a prompt injection attack? IBM. <https://www.ibm.com/think/topics/prompt-injection>
- Koutchme, C., Dainese, N., Sarsa, S., Hellas, A., Leinonen, J., & Denny, P. (2024). Open source language models can provide feedback: Evaluating LLMs' ability to help students using GPT-4-as-a-judge. arXiv preprint [arXiv:2405.05253](https://arxiv.org/abs/2405.05253). <https://arxiv.org/abs/2405.05253>
- Lester, B., Al-Rfou, R., & Constant, N. (2021). *The Power of Scale for Parameter-Efficient Prompt Tuning*. arXiv preprint [arXiv:2104.08691](https://arxiv.org/abs/2104.08691).
- Li, C. (2020, June 3). *OpenAI's GPT-3 Language Model: A Technical Overview*. Lambda Labs Blog. <https://lambda.ai/blog/demystifying-gpt-3>
- Li, X. L., & Liang, P. (2021). *Prefix-tuning: Optimizing continuous prompts for generation*. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL 2021)*. <https://arxiv.org/abs/2101.00190>
- Lin, C.-Y. (2004). *ROUGE: A package for automatic evaluation of summaries*. ACL Workshop on Text Summarization Branches Out. <https://aclanthology.org/W04-1013/>
- Liu, P., He, J., Chen, W., Yang, Y., Li, H., & Sun, M. (2023). *Lost in the middle: Assessing LLM context retention in extended inputs*. arXiv preprint [arXiv:2307.03172](https://arxiv.org/abs/2307.03172)
- Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., & Neubig, G. (2021). *Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing*. arXiv preprint [arXiv:2107.13586](https://arxiv.org/abs/2107.13586). <https://arxiv.org/abs/2107.13586>

- Madaan, A., Lin, Z., Sunkara, S., Gupta, P., Radev, D., & Mazaitis, K. (2023). *Self-Refine: Iterative re-finement with LLM feedback*. arXiv preprint [arXiv:2303.17651](https://arxiv.org/abs/2303.17651)
- Nadeem, M., Bethke, A., & Reddy, S. (2020). *StereoSet: Measuring stereotypical bias in pretrained language models*. arXiv preprint. <https://doi.org/10.48550/arXiv.2004.09456>
- OpenAI. (2022, March). *OpenAI API introduces GPT-3.5-turbo*. <https://openai.com/research/instruction-following>
- OpenAI. (2023). *GPT-4 Technical Report*. OpenAI. <https://arxiv.org/abs/2303.08774>
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. (2022). *Training language models to follow instructions with human feedback*. arXiv preprint. <https://arxiv.org/abs/2203.02155>
- Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). *BLEU: A method for automatic evaluation of machine translation*. Proceedings of the 40th annual meeting of the Association for Computational Linguistics, 311–318. <https://aclanthology.org/P02-1040/>
- Perez, E., Ribeiro, M. T., & Kiela, D. (2022). *Red teaming language models with language models*. arXiv preprint <https://arxiv.org/pdf/2202.03286>
- Reynolds, L., & McDonell, K. (2021). *Prompt programming for large language models: Beyond the few-shot paradigm*. arXiv preprint. <https://arxiv.org/abs/2102.07350>
- Schardt, D. (2023, March 20). ChatGPT is amazing. But beware its hallucinations! *Center for Science in the Public Interest*. <https://www.cspinet.org/blog/chatgpt-amazing-beware-its-hallucinations>
- Schulhoff, S. (2025, March 25). Prompt injection. *Learn Prompting*. https://learnprompting.org/docs/prompt_hacking/injection
- Sellam, T., Das, D., & Parikh, A. P. (2020). BLEURT: Learning robust metrics for text generation. *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 7881–7892. <https://aclanthology.org/2020.acl-main.704>
- Shin, T., Razeghi, Y., Logan IV, R. L., Wallace, E., & Singh, S. (2020). *AutoPrompt: Eliciting knowledge from language models with automatically generated prompts*. arXiv preprint [arXiv:2010.15980](https://arxiv.org/abs/2010.15980).
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention is all you need*. In *Advances in Neural Information Processing Systems* (pp. 5998-6008). <https://arxiv.org/abs/1706.03762>
- Warren, T. (2023, February 14). These are Microsoft's Bing AI secret rules and why it says it's named Sydney. *The Verge*. <https://www.theverge.com/23599441/microsoft-bing-ai-sydney-secret->
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Chi, E., Le, Q. V., & Zhou, D. (2022). *Chain of thought prompting elicits reasoning in large language models*. arXiv preprint. <https://arxiv.org/abs/2201.11903>

Zhang, T., Kishore, V., Wu, F., Weinberger, K., & Artzi, Y. (2020). *BERTScore: Evaluating text generation with BERT*. In *International Conference on Learning Representations (ICLR)*. [arXiv:1904.09675](https://arxiv.org/abs/1904.09675).

Zhu, L., Wang, X., & Wang, X. (2025). JudgeLM: Fine-tuned large language models are scalable judges. *arXiv preprint*. <https://arxiv.org/abs/2310.17631>

Appendices

Appendix 1. Dataset 1

```
[
  {
    "task": "summarization",
    "input": "Artificial intelligence is rapidly transforming industries by automating tasks, enabling new business models, and improving efficiency.",
    "reference": "AI is changing industries by improving efficiency and enabling new models."
  },
  {
    "task": "summarization",
    "input": "Renewable energy sources like solar and wind are gaining popularity due to environmental concerns and declining costs.",
    "reference": "Solar and wind energy are growing due to cost and environmental benefits."
  },
  {
    "task": "summarization",
    "input": "Cloud computing allows businesses to scale operations and access computing resources without heavy infrastructure investment.",
    "reference": "Cloud computing supports scalability without high infrastructure costs."
  },
  {
    "task": "classification",
    "input": "I absolutely loved the new design of the website—clean and easy to navigate.",
    "reference": "Positive"
  },
  {
    "task": "classification",
    "input": "It was okay, nothing special. I wouldn't go out of my way to recommend it.",
    "reference": "Neutral"
  },
  {
    "task": "classification",
    "input": "The app crashed multiple times and was incredibly frustrating to use.",
    "reference": "Negative"
  },
  {
    "task": "paraphrasing",
    "input": "How can I enhance my resume for a career in marketing?",
    "reference": "What are some ways to improve my CV for a marketing role?"
  },
  {
    "task": "paraphrasing",
    "input": "She didn't like the movie because the plot was too predictable.",
    "reference": "She found the movie uninteresting due to its predictable storyline."
  },
]
```

```

{
  "task": "paraphrasing",
  "input": "Learning to code can open up many career opportunities in tech.",
  "reference": "Acquiring coding skills can lead to various job prospects in the tech field."
},
{
  "task": "classification",
  "input": "The new update fixed all the bugs. I'm really happy with the performance now.",
  "reference": "Positive"
},
{
  "task": "classification",
  "input": "The customer service was okay. Nothing special, but nothing terrible either.",
  "reference": "Neutral"
},
{
  "task": "classification",
  "input": "The delivery was late, and the packaging was damaged. Very disappointing.",
  "reference": "Negative"
},
{
  "task": "paraphrasing",
  "input": "I need to find a better way to manage my time during the workday.",
  "reference": "I'm looking for an improved method to organize my workday schedule."
},
{
  "task": "paraphrasing",
  "input": "The seminar was informative and covered the topic in depth.",
  "reference": "The workshop provided detailed and insightful information on the topic."
},
{
  "task": "paraphrasing",
  "input": "Can you help me understand how this algorithm works?",
  "reference": "Could you explain the functioning of this algorithm to me?"
}
]

```

Appendix 2. Dataset 2

```
[
  {
    "task": "summarization",
    "input": "Renewable energy like solar and wind has become cheaper, leading to widespread adoption across the globe.",
    "reference": "Falling costs are driving global use of renewable energy.",
    "complexity": "basic",
    "domain": "education"
  },
  {
    "task": "summarization",
    "input": "The human brain processes complex visual data rapidly, which helps in navigation and spatial awareness.",
    "reference": "Visual data processing aids brain navigation.",
    "complexity": "complex",
    "domain": "health"
  },
  {
    "task": "summarization",
    "input": "Renewable energy like solar and wind has become cheaper, leading to widespread adoption across the globe.",
    "reference": "Falling costs are driving global use of renewable energy.",
    "complexity": "complex",
    "domain": "finance"
  },
  {
    "task": "summarization",
    "input": "AI is transforming industries by automating routine tasks and enhancing decision-making through data analysis.",
    "reference": "AI automates tasks and supports decisions through data.",
    "complexity": "basic",
    "domain": "education"
  },
  {
    "task": "summarization",
    "input": "Renewable energy like solar and wind has become cheaper, leading to wide-spread adoption across the globe.",
    "reference": "Falling costs are driving global use of renewable energy.",
    "complexity": "moderate",
    "domain": "tech"
  },
  {
    "task": "summarization",
    "input": "AI is transforming industries by automating routine tasks and enhancing decision-making through data analysis.",
    "reference": "AI automates tasks and supports decisions through data.",

```



```

    "complexity": "complex",
    "domain": "education"
  },
  {
    "task": "summarization",
    "input": "The human brain process-es complex visual data rapidly, which helps in navigation and spatial awareness.",
    "reference": "Visual data processing aids brain navigation.",
    "complexity": "basic",
    "domain": "tech"
  },
  {
    "task": "summarization",
    "input": "Renewable energy like solar and wind has become cheaper, leading to widespread adoption across the globe.",
    "reference": "Falling costs are driving global use of renewable energy.",
    "complexity": "basic",
    "domain": "finance"
  },
  {
    "task": "summarization",
    "input": "AI is transforming industries by automating routine tasks and enhancing decision-making through data analysis.",
    "reference": "AI automates tasks and supports decisions through data.",
    "complexity": "moderate",
    "domain": "general"
  },
  {
    "task": "classification",
    "input": "Extremely disappointed. The quality was terrible.",
    "reference": "Negative",
    "complexity": "basic",
    "domain": "tech"
  },
  {
    "task": "classification",
    "input": "This product exceeded my expectations. I'd absolutely buy it again!",
    "reference": "Positive",
    "complexity": "basic",
    "domain": "tech"
  },
  {
    "task": "classification",
    "input": "Extremely disappointed. The quality was terrible.",
    "reference": "Negative",
    "complexity": "basic",
    "domain": "tech"
  },
  {

```

```

"task": "classification",
"input": "This product exceeded my expectations. I'd absolutely buy it again!",
"reference": "Positive",
"complexity": "moderate",
"domain": "tech"
},
{
"task": "classification",
"input": "Extremely disappointed. The quality was terrible.",
"reference": "Negative",
"complexity": "complex",
"domain": "general"
},
{
"task": "classification",
"input": "This product exceeded my expectations. I'd absolutely buy it again!",
"reference": "Positive",
"complexity": "complex",
"domain": "finance"
},
{
"task": "classification",
"input": "This product exceeded my expectations. I'd absolutely buy it again!",
"reference": "Positive",
"complexity": "basic",
"domain": "education"
},
{
"task": "classification",
"input": "This product exceeded my expectations. I'd absolutely buy it again!",
"reference": "Positive",
"complexity": "moderate",
"domain": "tech"
},
{
"task": "classification",
"input": "It was fine. Not the best, not the worst.",
"reference": "Neutral",
"complexity": "moderate",
"domain": "finance"
},
{
"task": "paraphrasing",
"input": "The research suggests that exercise enhances cognitive performance.",
"reference": "According to studies, physical activity improves brain function.",
"complexity": "basic",
"domain": "general"
},
{

```

```

"task": "paraphrasing",
"input": "The research suggests that exercise enhances cognitive performance.",
"reference": "According to studies, physical activity improves brain function.",
"complexity": "complex",
"domain": "education"
},
{
  "task": "paraphrasing",
  "input": "I need to im-prove how I manage my time.",
  "reference": "I must find a better way to organize my schedule.",
  "complexity": "basic",
  "domain": "education"
},
{
  "task": "paraphrasing",
  "input": "I need to improve how I manage my time.",
  "reference": "I must find a better way to organize my schedule.",
  "complexity": "moderate",
  "domain": "education"
},
{
  "task": "paraphrasing",
  "input": "Could you explain the concept again in a simpler way?",
  "reference": "Can you simplify and explain it once more?",
  "complexity": "complex",
  "domain": "education"
},
{
  "task": "paraphrasing",
  "input": "Could you explain the concept again in a simpler way?",
  "reference": "Can you simplify and explain it once more?",
  "complexity": "basic",
  "domain": "education"
},
{
  "task": "paraphrasing",
  "input": "Could you explain the concept again in a simpler way?",
  "reference": "Can you simplify and explain it once more?",
  "complexity": "moderate",
  "domain": "general"
},
{
  "task": "paraphrasing",
  "input": "I need to improve how I manage my time.",
  "reference": "I must find a better way to organize my schedule.",
  "complexity": "moderate",
  "domain": "tech"
},
{

```

```
"task": "paraphrasing",  
"input": "The research suggests that exercise enhances cognitive performance.",  
"reference": "According to studies, physical activity improves brain function.",  
"complexity": "complex",  
"domain": "health"  
}  
]
```

Appendix 3. Dataset 3

```
[
  {
    "task": "summarization",
    "input": "Electric vehicles are gaining popularity due to lower emissions and advancing battery technologies.",
    "reference": "EVs are growing in popularity because of battery tech and lower emissions.",
    "complexity": "basic",
    "domain": "general"
  },
  {
    "task": "summarization",
    "input": "The COVID-19 pandemic has accelerated digital transformation across healthcare and education sectors.",
    "reference": "COVID-19 sped up digital adoption in health and education.",
    "complexity": "moderate",
    "domain": "general"
  },
  {
    "task": "summarization",
    "input": "Electric vehicles are gaining popularity due to lower emissions and advancing battery technologies.",
    "reference": "EVs are growing in popularity because of battery tech and lower emissions.",
    "complexity": "complex",
    "domain": "general"
  },
  {
    "task": "summarization",
    "input": "Electric vehicles are gaining popularity due to lower emissions and advancing battery technologies.",
    "reference": "EVs are growing in popularity because of battery tech and lower emissions.",
    "complexity": "basic",
    "domain": "education"
  },
  {
    "task": "summarization",
    "input": "Electric vehicles are gaining popularity due to lower emissions and advancing battery technologies.",
    "reference": "EVs are growing in popularity because of battery tech and lower emissions.",
    "complexity": "moderate",
    "domain": "education"
  },
  {
    "task": "summarization",
    "input": "Electric vehicles are gaining popularity due to lower emissions and advancing battery technologies.",
    "reference": "EVs are growing in popularity because of battery tech and lower emissions.",
```

```

    "complexity": "complex",
    "domain": "tech"
  },
  {
    "task": "summarization",
    "input": "The COVID-19 pandemic has accelerated digital trans-formation across healthcare and
education sectors.",
    "reference": "COVID-19 sped up digital adoption in health and education.",
    "complexity": "basic",
    "domain": "finance"
  },
  {
    "task": "summarization",
    "input": "The COVID-19 pandemic has accelerated digital transformation across healthcare and
education sectors.",
    "reference": "COVID-19 sped up digital adoption in health and education.",
    "complexity": "moderate",
    "domain": "tech"
  },
  {
    "task": "summarization",
    "input": "The COVID-19 pandemic has accelerated digital transformation across healthcare and
education sectors.",
    "reference": "COVID-19 sped up digital adoption in health and education.",
    "complexity": "complex",
    "domain": "health"
  },
  {
    "task": "classification",
    "input": "I'm not impressed with the qual-ity at all.",
    "reference": "Negative",
    "complexity": "basic",
    "domain": "health"
  },
  {
    "task": "classification",
    "input": "The app works fine but lacks some key features.",
    "reference": "Neutral",
    "complexity": "moderate",
    "domain": "health"
  },
  {
    "task": "classification",
    "input": "I'm not impressed with the quality at all.",
    "reference": "Negative",
    "complexity": "complex",
    "domain": "health"
  },
  {

```

```

"task": "classification",
"input": "The app works fine but lacks some key features.",
"reference": "Neutral",
"complexity": "basic",
"domain": "health"
},
{
  "task": "classification",
  "input": "This is exactly what I needed. Great tool!",
  "reference": "Positive",
  "complexity": "moderate",
  "domain": "finance"
},
{
  "task": "classification",
  "input": "The app works fine but lacks some key features.",
  "reference": "Neutral",
  "complexity": "complex",
  "domain": "general"
},
{
  "task": "classification",
  "input": "I'm not impressed with the quality at all.",
  "reference": "Negative",
  "complexity": "basic",
  "domain": "finance"
},
{
  "task": "classification",
  "input": "The app works fine but lacks some key features.",
  "reference": "Neutral",
  "complexity": "moderate",
  "domain": "finance"
},
{
  "task": "classification",
  "input": "This is exactly what I needed. Great tool!",
  "reference": "Positive",
  "complexity": "com-plex",
  "domain": "education"
},
{
  "task": "paraphrasing",
  "input": "Please review the attached file before our discussion.",
  "reference": "Check the attached file prior to the meeting.",
  "complexity": "basic",
  "domain": "education"
},
{

```

```

"task": "paraphrasing",
"input": "Please review the attached file before our discussion.",
"reference": "Check the attached file prior to the meeting.",
"complexity": "moderate",
"domain": "tech"
},
{
  "task": "paraphrasing",
  "input": "He couldn't attend the meeting due to a scheduling conflict.",
  "reference": "A scheduling issue prevented him from attending.",
  "complexity": "complex",
  "domain": "general"
},
{
  "task": "paraphrasing",
  "input": "Please review the attached file before our discussion.",
  "reference": "Check the attached file prior to the meet-ing.",
  "complexity": "basic",
  "domain": "general"
},
{
  "task": "paraphrasing",
  "input": "He couldn't attend the meeting due to a scheduling conflict.",
  "reference": "A scheduling issue prevented him from attending.",
  "complexity": "moderate",
  "domain": "general"
},
{
  "task": "paraphrasing",
  "in-put": "He couldn't attend the meeting due to a scheduling conflict.",
  "reference": "A scheduling issue prevented him from attending.",
  "complexity": "complex",
  "domain": "tech"
},
{
  "task": "paraphrasing",
  "input": "He couldn't attend the meeting due to a scheduling conflict.",
  "reference": "A scheduling issue prevented him from attending.",
  "complexity": "basic",
  "domain": "education"
},
{
  "task": "paraphrasing",
  "input": "Please review the attached file before our discussion.",
  "reference": "Check the attached file prior to the meeting.",
  "complexity": "moderate",
  "domain": "tech"
},
{

```



```
"task": "paraphrasing",  
"input": "He couldn't attend the meeting due to a scheduling conflict.",  
"reference": "A scheduling issue prevented him from attending.",  
"complexity": "complex",  
"domain": "general"  
}  
]
```